Lecture Note 8: Introduction to Raspberry Pi

CSS452 Internet of Things

Dr. Seksan Laitrakun
School of ICT, SIIT, Thammasat University

Raspberry Pi

A Raspberry Pi is not a microcontroller; it is a powerful Single-Board Computer (SBC) that serves a unique role in IoT systems.

1) A Bridge Between Software and Hardware:
   - It runs a full operating system (Linux), allowing for complex software development.
   - It features GPIO pins to directly interface with sensors and actuators, just like a microcontroller.
2) Powerful Processing Capabilities:
   - It can run demanding applications that are too heavy for a typical microcontroller, such as: Web servers, Databases, Python scripts with complex libraries, Machine learning models.
3) An Ideal IoT Gateway:
   - Its combination of robust networking (Wi-Fi, Ethernet), processing power, and ability to run software like an MQTT broker makes it a perfect choice for a central hub or gateway in an IoT system.

Outlines

1. Hardware and Pins
2. Raspberry Pi Setup
3. Programming GPIO Pins using gpiozero Python Library
4. Creating a Simple GUI using guizero Python Library
5. Programming Raspberry Pi Towards IoT Applications Using Python

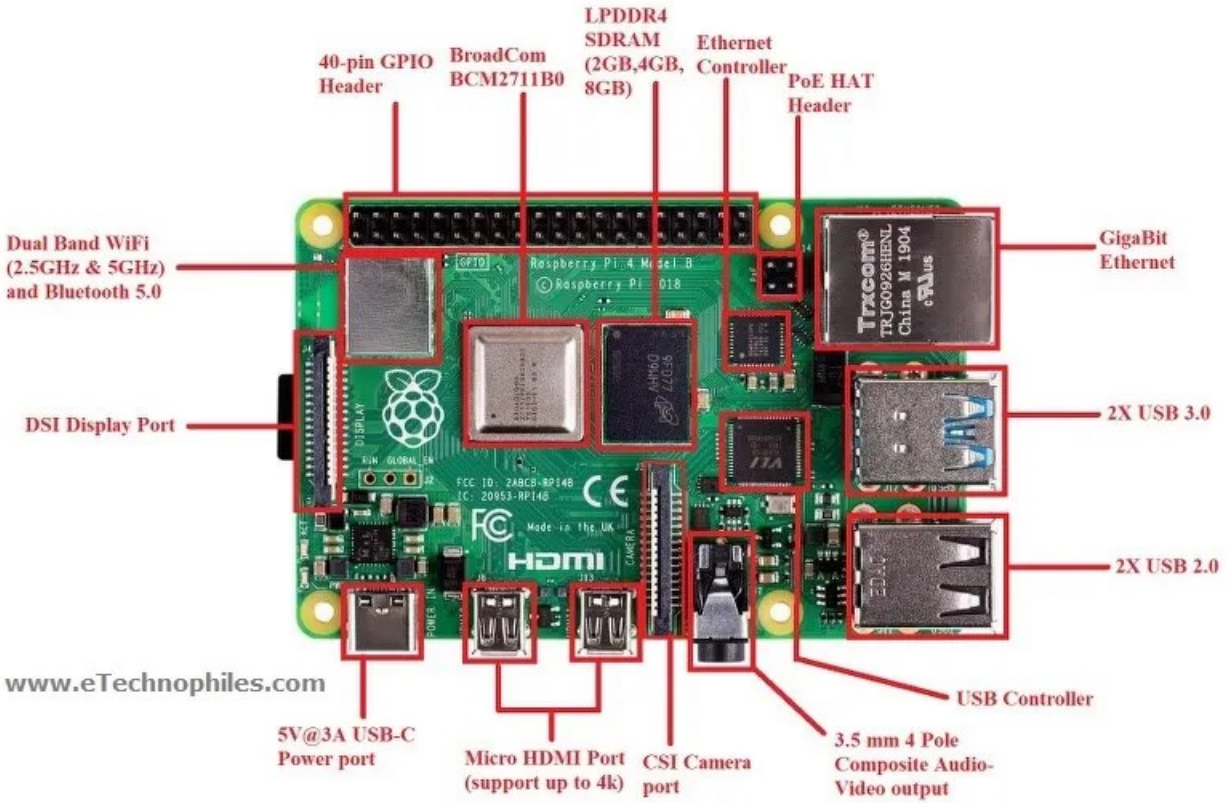
References

- Web Pages:
  - Raspberry Pi 4 Layout: https://www.etechnophiles.com/raspberry-pi-4-gpio-pinout-specifications-and-schematic/
  - Raspberry Pi 4 Pinout, Features and Peripherals: https://microcontrollerslab.com/raspberry-pi-4-pinout-description-features-peripherals-applications/
  - gpiozero Library: https://gpiozero.readthedocs.io/en/stable/index.html#
  - guizero Library: https://lawsie.github.io/guizero/
- Books:
  - Chapters 12 – 18, 20 of "Raspberry Pi 4: The Ultimate Step-by-Step" by Liam Clark (2021).
  - Chapter 4 of "Learn Electronics with Raspberry Pi," 2nd Ed., by Stewart Watkiss (2020).
  - Chapters 10, 16 of "Raspberry Pi Cookbook," 3rd Ed., by Simon Monk (2019).

1. Hardware and Pins

Raspberry Pi 4 Hardware

Raspberry Pi is a small-sized computer used Linux operating system and has pins to connect with sensors and actuators. It is mini size computer used mostly to run larger and smart programs to achieve output quickly.

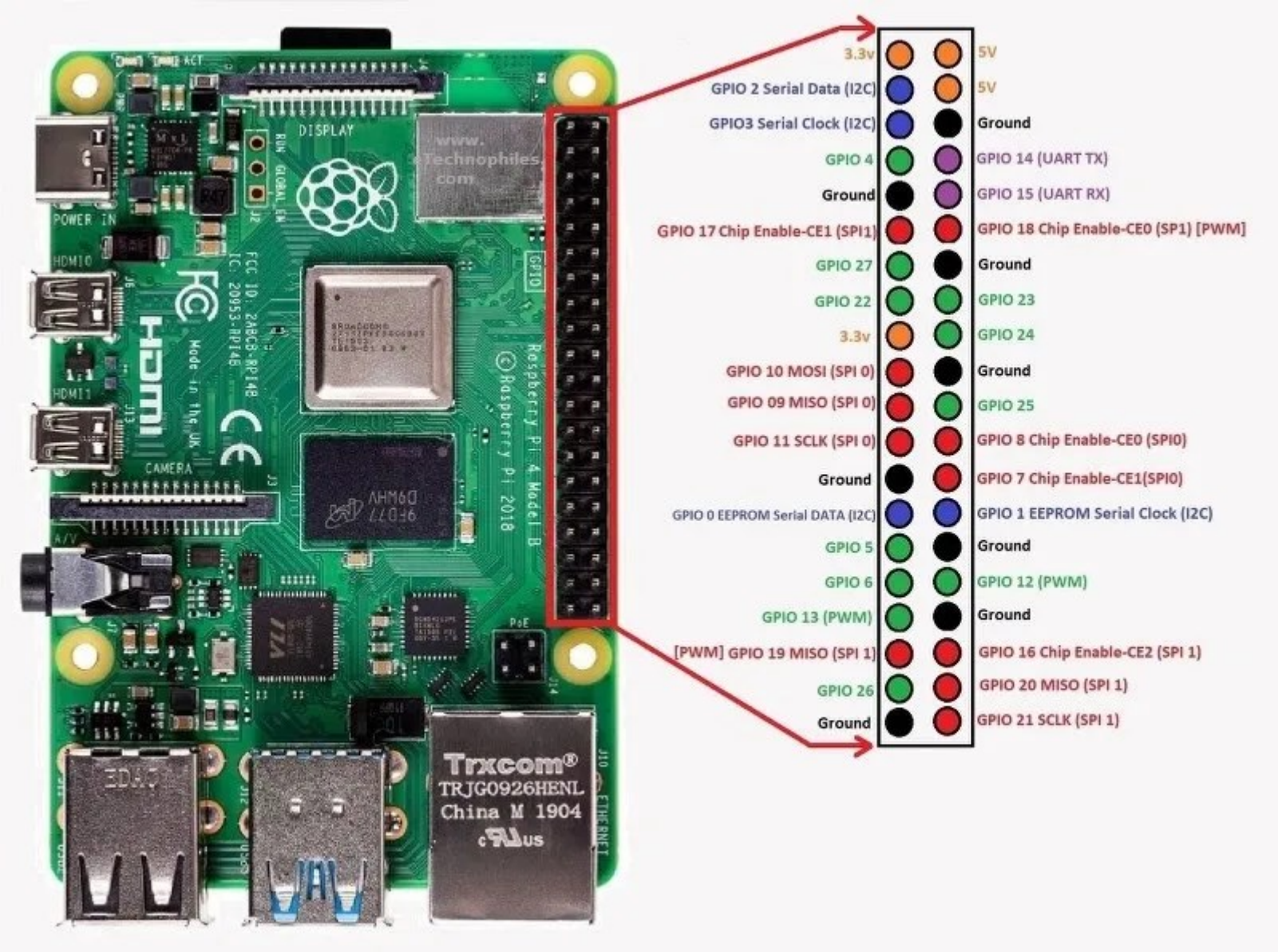




Raspberry Pi 4 Hardware

- CPU: It consists of a Broadcom BCM2711 chip which contains a 1.5GHz 64-bit quad-core ARM Cortex-A53 processor (using an ARMv8-architecture core).
- GPU: Broadcom VideoCore VI @ 500 MHz was released in 2009. It is capable of BluRay quality video playback, H.265 (4Kp60 decode); H.264 (1080p60 decode, 1080p30 encode); OpenGL ES, 3.0 graphics.
- RAM: It comes with 2GB, 4GB, and 8GB (depends on different versions) variants of LPDDR4 SDRAM.
- USB port: It consists of two USB 3.0 and two USB 2.0 ports to connect it to an external keyboard, mouse, or other peripheral devices.
- USB power port: It consists of a 5.1V, 3A USB type-C power port.
- HDMI port: Two micro HDMI ports capable of supporting up to 4k@60HZ resolution.
- Ethernet Port: It comes with true Gigabit Ethernet capable of sending Ethernet frames at a rate of one gigabit per second (1 billion bits per second).
- Composite Video Output: Both the audio output socket and the video composite socket reside in a single 4-pole 3.5mm socket.
- SD card Slot: A micro-SD card slot is used for booting up the operating system and storage purposes.

Raspberry Pi 4 Hardware

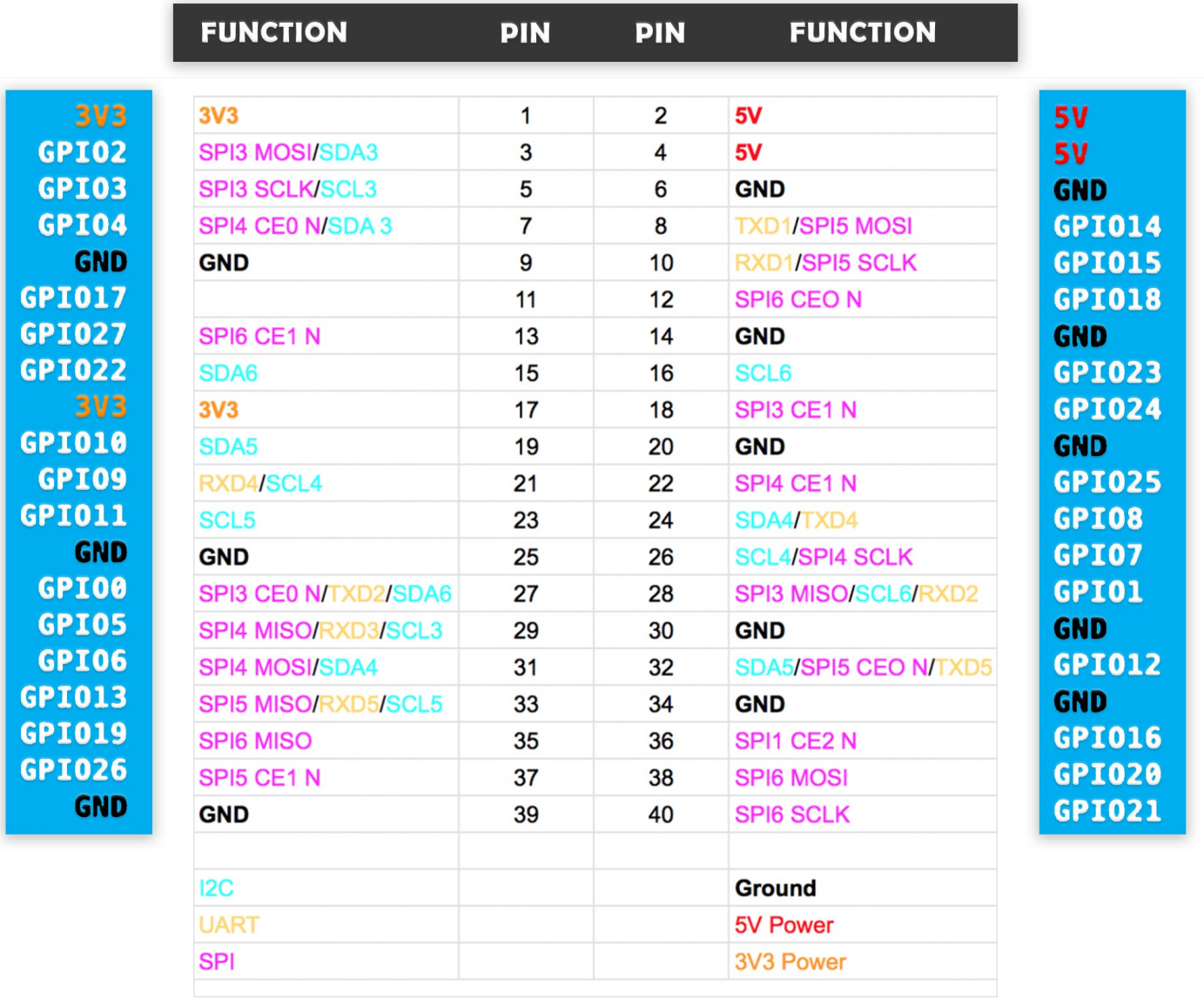


- 40 pins:
  - 26 GPIO pins,
  - two 5V pins,
  - two 3V3 pins,
  - 7 ground pins (0V),
  - other GPIO pins of RPI 4 are capable of generating PWM output and the board supports SPI, I2C, and UART serial communication protocols.

Raspberry Pi 4 Pins

We connect input/output devices to Raspberry-Pi pins. Here is a complete overview with all the GPIOs and their primary function.

- Each pin has a physical pin number: 1, 2, 3, ….
- We have power supply pins (3.3V and 5V) and ground pins (0V).
- We have 26 pins to connect with input/output devices i.e., the GPIO pins.
  - In our Python programs, we refer a pin by its GPIO number not its physical pin number.



	FUNCTION	PIN	PIN	FUNCTION	
3V3	3V3	1	2	5V	5V
GPIO2	SPI3 MOSI/SDA3	3	4	5V	5V
GPIO3	SPI3 SCLK/SCL3	5	6	GND	GND
GPIO4	SPI4 CE0 N/SDA 3	7	8	TXD1/SPI5 MOSI	GPIO14
GND	GND	9	10	RXD1/SPI5 SCLK	GPIO15
GPIO17		11	12	SPI6 CEO N	GPIO18
GPIO27	SPI6 CE1 N	13	14	GND	GND
GPIO22	SDA6	15	16	SCL6	GPIO23
3V3	3V3	17	18	SPI3 CE1 N	GPIO24
GPIO10	SDA5	19	20	GND	GND
GPIO9	RXD4/SCL4	21	22	SPI4 CE1 N	GPIO25
GPIO11	SCL5	23	24	SDA4/TXD4	GPIO8
GND	GND	25	26	SCL4/SPI4 SCLK	GPIO7
GPIO0	SPI3 CE0 N/TXD2/SDA6	27	28	SPI3 MISO/SCL6/RXD2	GPIO1
GPIO5	SPI4 MISO/RXD3/SCL3	29	30	GND	GND
GPIO6	SPI4 MOSI/SDA4	31	32	SDA5/SPI5 CEO N/TXD5	GPIO12
GPIO13	SPI5 MISO/RXD5/SCL5	33	34	GND	GND
GPIO19	SPI6 MISO	35	36	SPI1 CE2 N	GPIO16
GPIO26	SPI5 CE1 N	37	38	SPI6 MOSI	GPIO20
GND	GND	39	40	SPI6 SCLK	GPIO21
	I2C			Ground	
	UART			5V Power	
	SPI			3V3 Power	

Raspberry Pi 4 Pins

- Referring to a pin by its GPIO number is known as the BCM (Broadcom) numbering system, which will be used in our Python programs.
- Some pins have alternative functions (being used for communications e.x, I2C, SPI, …).
- Some references mentioned that Pins 27 and 28 are reserved (no used).
- There are 8 ground pins (GND).
- The GPIO pins of Raspberry Pi 4 all are digital pins. They can be either input pins or output pins.
  - Input voltage between 1.8V and 3.3V is read as HIGH by the Raspberry Pi. And when the input voltage is lower than 1.8V, it is read as LOW.
  - Do not connect an external device with an output voltage above 3.3V to any of the GPIO pins, or else it will burn your Raspberry Pi board.
- PWM (pulse-width modulation) pins:
  - Software PWM is available on all pins.
  - Hardware PWM is available on these pins only: GPIO12, GPIO13, GPIO18, GPIO19.

2. Raspberry Pi Setup

Very Important:

- You will set up a couple of user names and passwords. Make sure that you record all of them since you will need them to access your Raspberry Pi, again and again.

1. Raspberry Pi OS

Raspberry Pi need an operating system which must be installed in an SD card. In our course, you have been given an SD card with the Raspberry Pi OS.

- You do not need to do the OS installation.
- Note that the default user name is *pi* and the default password is *raspberry*

Installing Raspberry Pi OS (FYI)

If you prefer to install the OS in the SD card by your own, feel free to do so. The Raspberry Pi OS can be installed as follows.

- We can install an operating system in an SD card easily by using the Raspberry Pi Imager. The Raspberry Pi Imager can be downloaded from this:
  https://www.raspberrypi.org/software/

- By running the Raspberry Pi Imager, we can select an operating system and install it in an SD card easily. A demonstration video is shown here:
  https://www.youtube.com/watch?v=ntaXWS8Lk34

- In this course, we install "Raspberry Pi OS".
- Now, you can run your Raspberry Pi by inserting this SD card to the Raspberry Pi and power it up.

2. Access Raspberry Pi via a Computer

- During the class, to work on Raspberry Pi, we might need to use a computer notebook's keyboard, mouse, and monitor to access the Raspberry Pi.

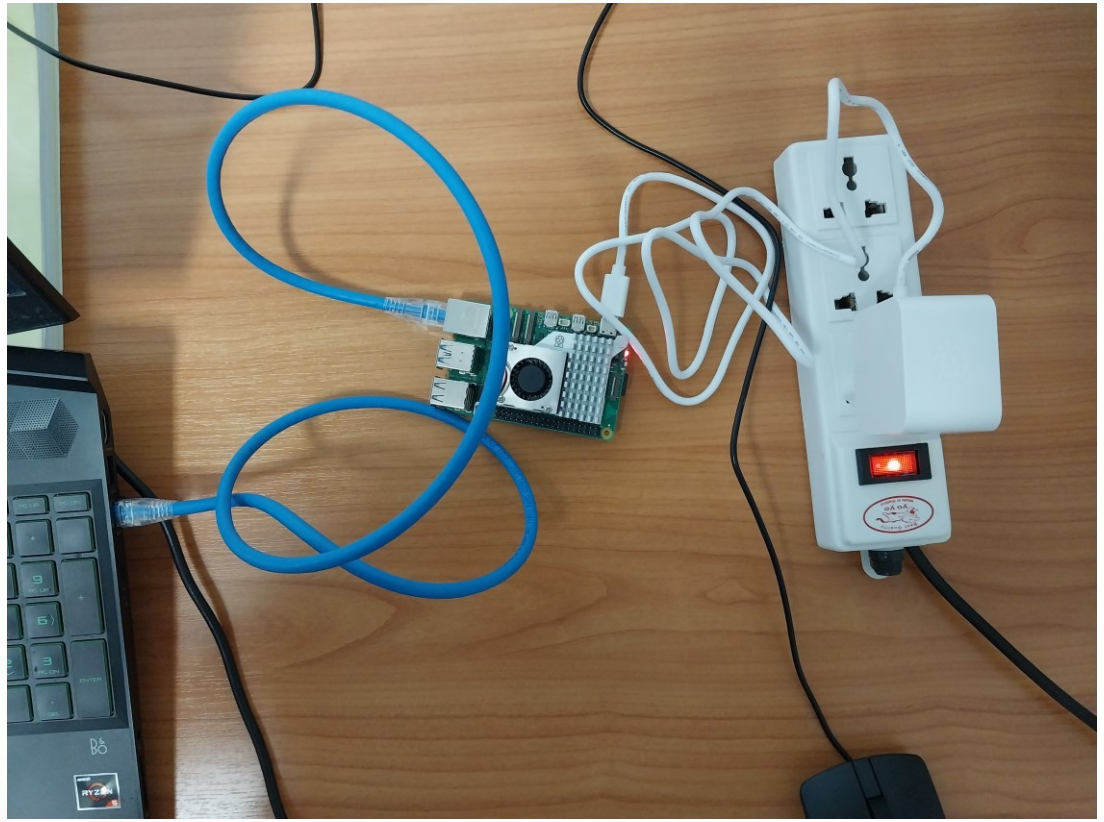
- We will access a Raspberry Pi via an ethernet cable. The following software will be used. Please install them on your computer notebook:
  - Putty (https://putty.org/index.html)
  - VNC Viewer (https://vnc-viewer.en.softonic.com/, www.realvnc.com)

2. Access Raspberry Pi via a Computer

- We have the following steps as shown in the next pages for setup. These steps are from the these references:
  - https://beebom.com/how-use-windows-laptop-as-monitor-raspberry-pi/
  - https://blackdevice.com/how-to-connect-to-raspberry-pi/#:~:text=Direct%20ethernet%20connection,%C2%BB%20menu%2C%20and%20click%20Ok.
  - https://raspberrypi-guide.github.io/networking/create-direct-ethernet-connection#:~:text=a%20Windows%20PC-,Direct%20connection%20with%20a%20Mac,without%20the%20ethernet%20cable%20again.

2. Access Raspberry Pi via a Computer

1) Power on your Raspberry Pi (with SD card inside).
2) Connect the cable: Use an Ethernet cable to connect the Raspberry Pi directly to your computer's Ethernet port, or to a USB-to-Ethernet adapter if your computer lacks one.



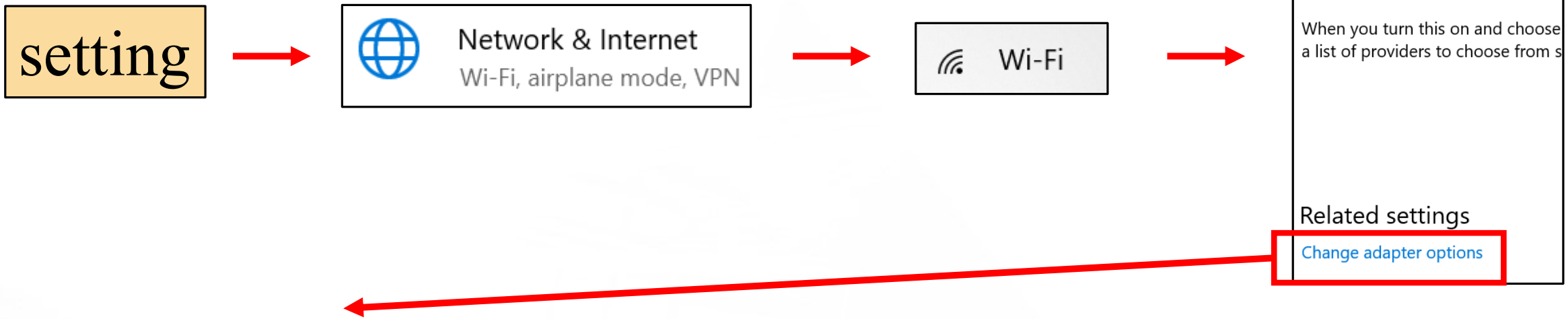
2. Access Raspberry Pi via a Computer

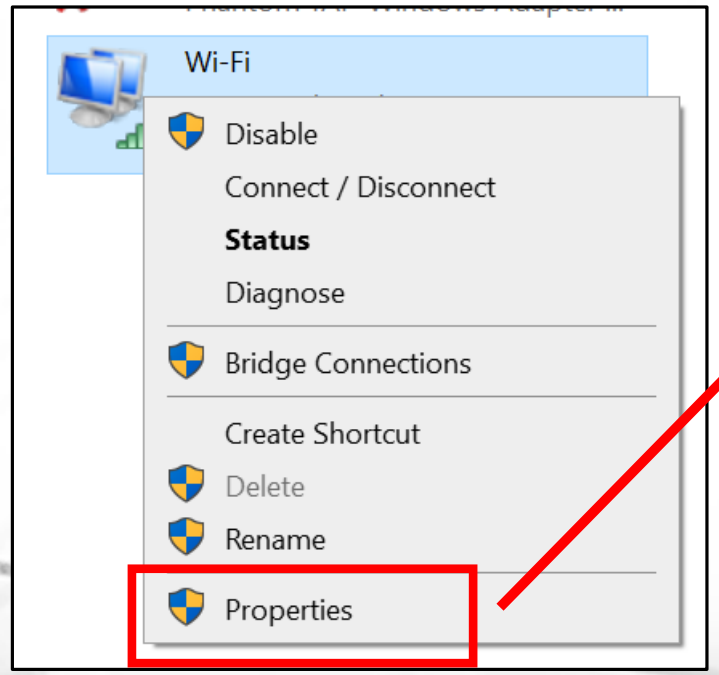
3) Share the internet connection:

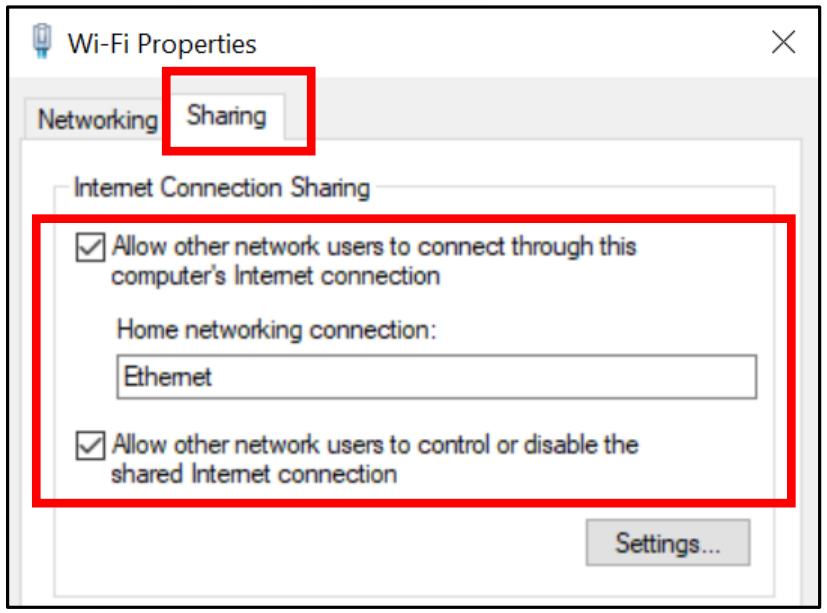
3.1) On Windows:

- Go to “Settings” > “Network & Internet” > “Wi-Fi” and click “Change adapter options”.
- Right-click your internet-connected adapter (e.g., Wi-Fi), go to “Properties” > “Sharing”, and check the box to “Allow other network users to connect”.
- Under the “Home networking connection”, select the “Ethernet” connection to the Pi from the “Home networking connection” dropdown and click “OK”.

2. Access Raspberry Pi via a Computer







2. Access Raspberry Pi via a Computer

3.2) On macOS:

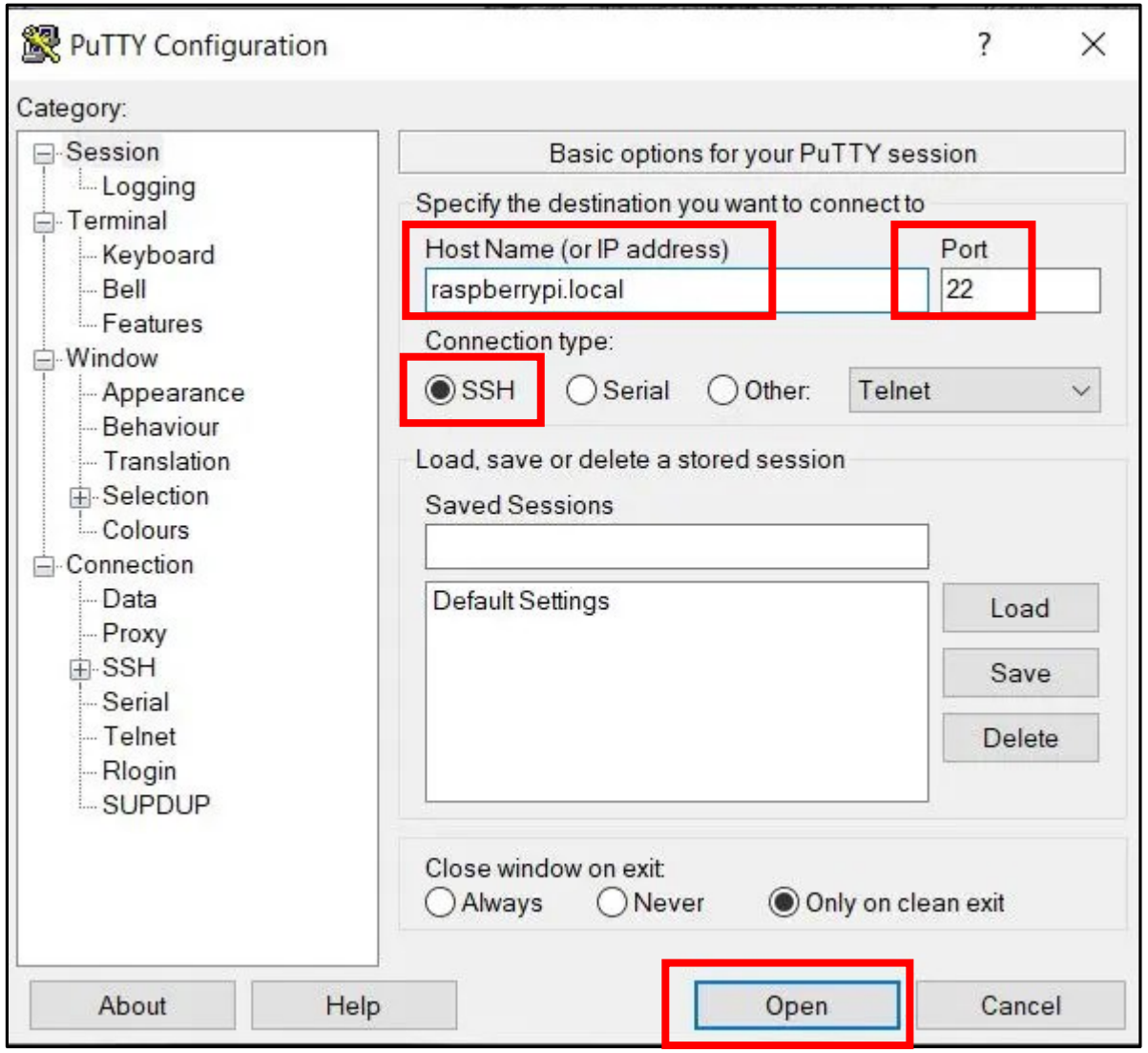
- Go to “System Preferences” > “Sharing”.
- Select “Internet Sharing” from the list on the left.
- From the “Share your connection from” dropdown, select your internet source (like Wi-Fi). Below, check the box next to your wired network interface (e.g., USB 10/100/1000 LAN).

2. Access Raspberry Pi via a Computer

4. Open Putty:
   - Type raspberrypi.local in the Host Name field.
   - Select "SSH" as the Connection type.
   - Click on "Open" at the bottom.

Remark:

- If you cannot access the Raspberry Pi via SSH, your Raspberry Pi might miss the file named "ssh".
- Then, you might need to create this file "ssh" in your Pi's SD card.
- See Step 2 in this reference: https://beebom.com/how-use-windows-laptop-as-monitor-raspberry-pi/



2. Access Raspberry Pi via a Computer

5. A console window will then open up. By default,
   - Raspberry Pi's username: pi
   - Password: raspberry

   Type these details on the console window and press the Enter. You will successfully log into Raspberry Pi's console.



6. To config the Raspberry Pi, type the command:

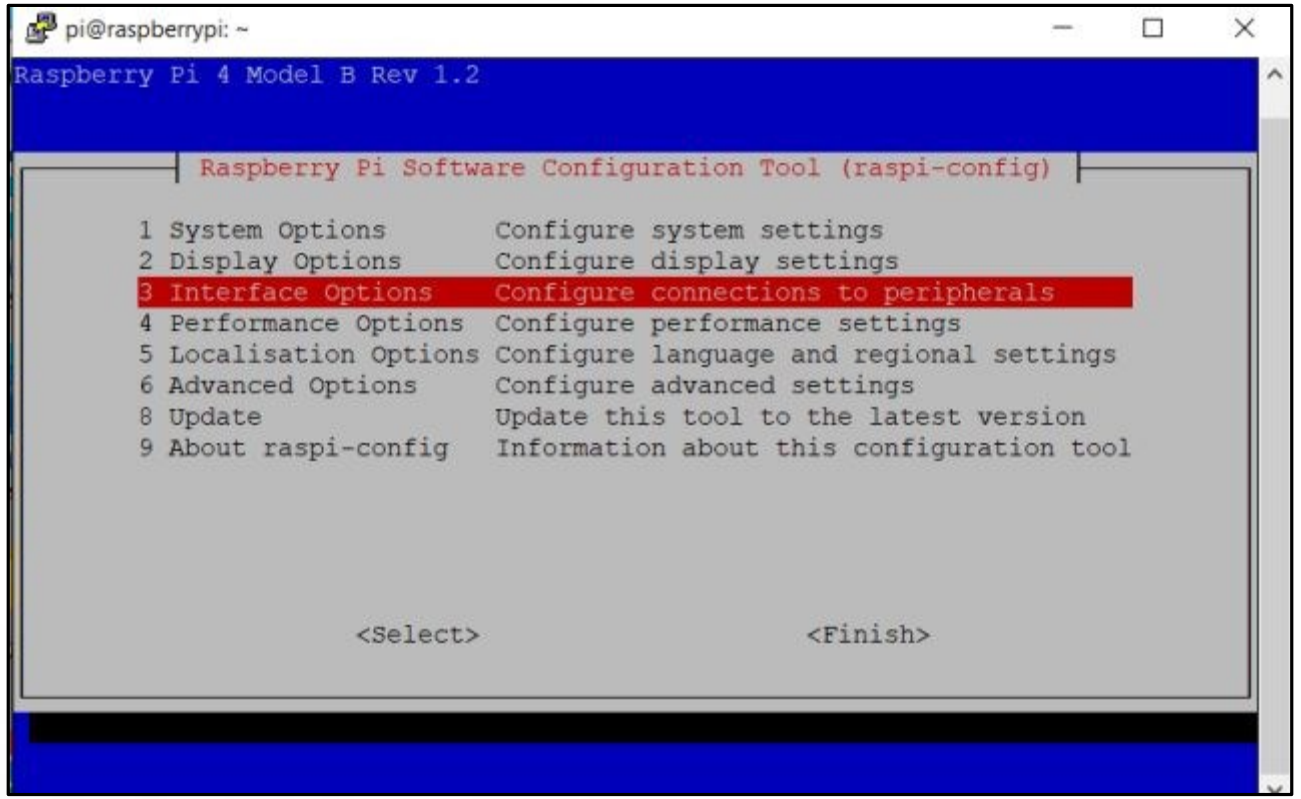
   sudo raspi-config

   and press the Enter.



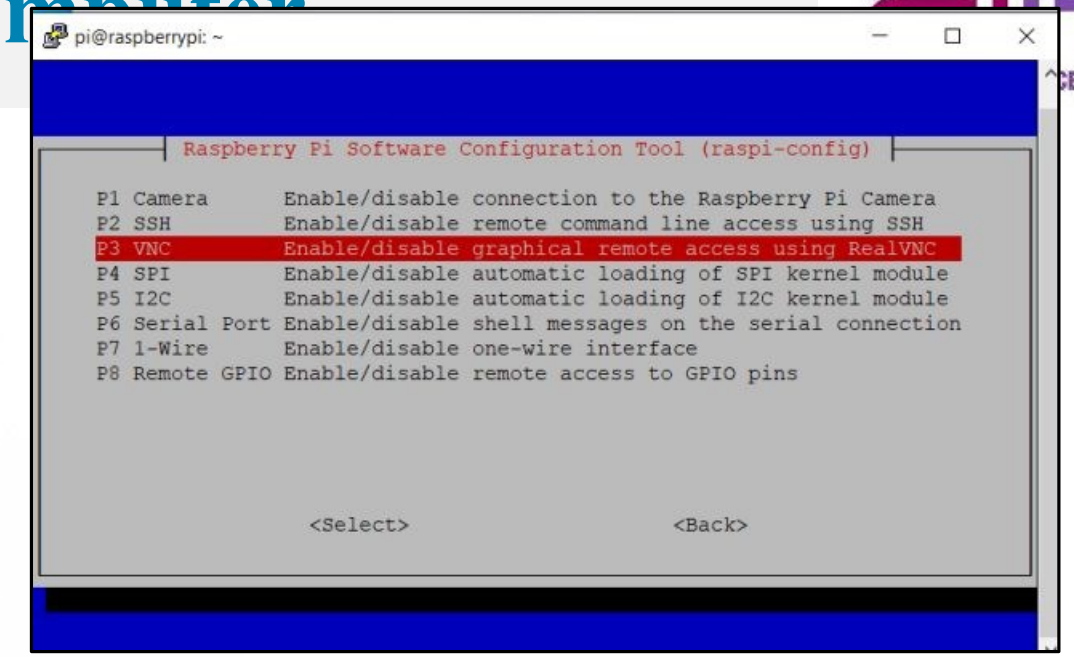
2. Access Raspberry Pi via a Computer

7. It will launch the configuration tool for Raspberry Pi.
   - Move to “Interface Options” or “Interfacing Options” using the keyboard’s arrow keys.
   - Then, press the Enter.



2. Access Raspberry Pi via a Computer

8. Under Interface options, move to "**VNC**" and press the Enter.

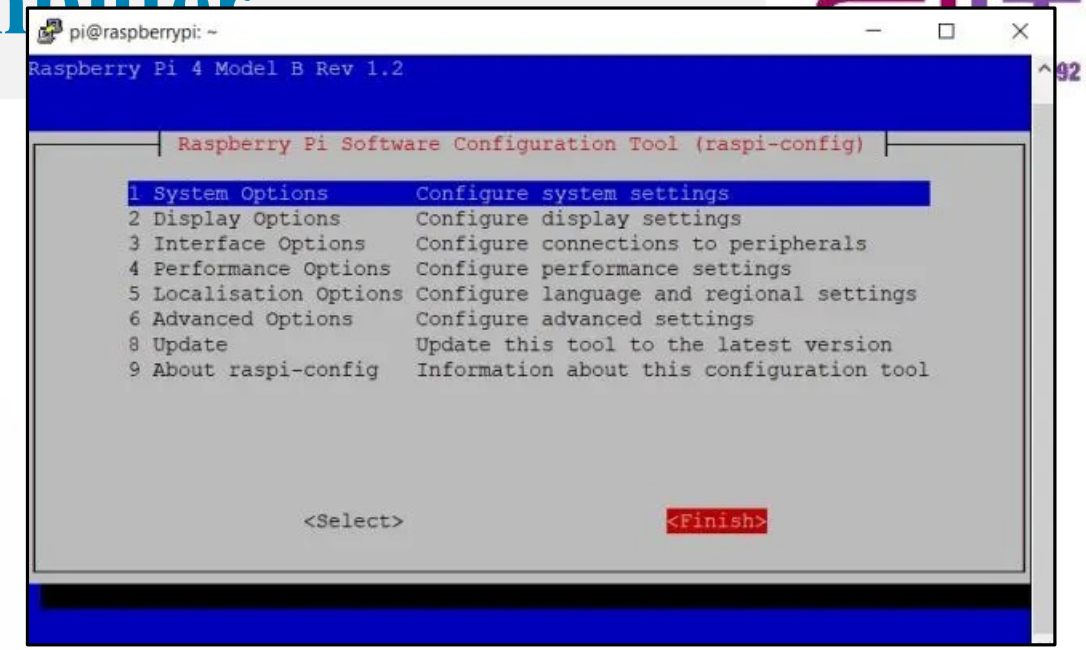


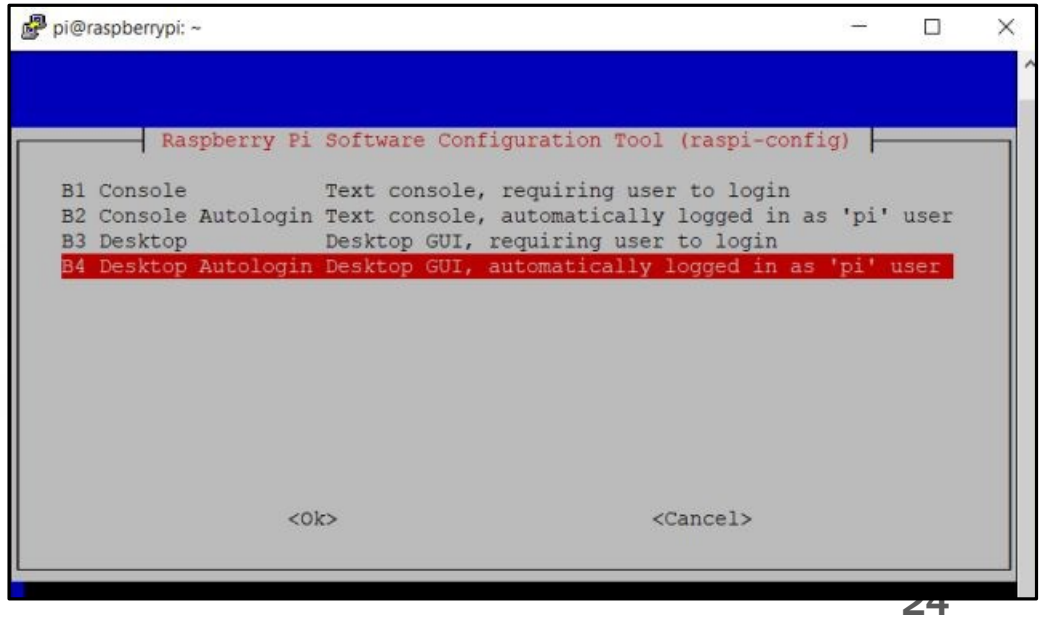
9. On the next page, choose "Yes" and press the Enter.



2. Access Raspberry Pi via a Computer

10. Open “System Options”
11. Go to “Boot/ Auto Login”.
12. Next, move to the “Desktop Autologin” option. (If you are using an old Raspberry Pi, navigate to Boot “Options -> Desktop/ CLI -> Desktop Autologin”.)
13. Select Ok button.

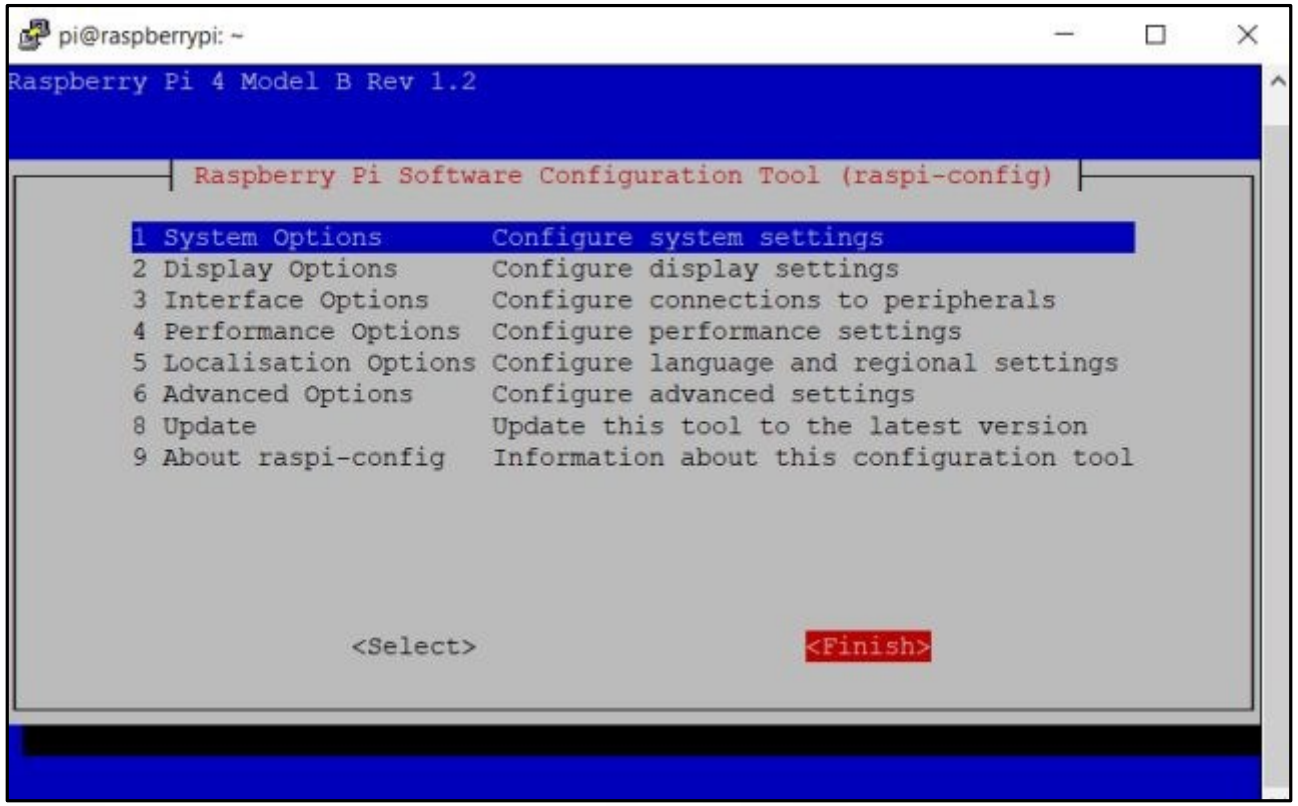




2. Access Raspberry Pi via a Computer

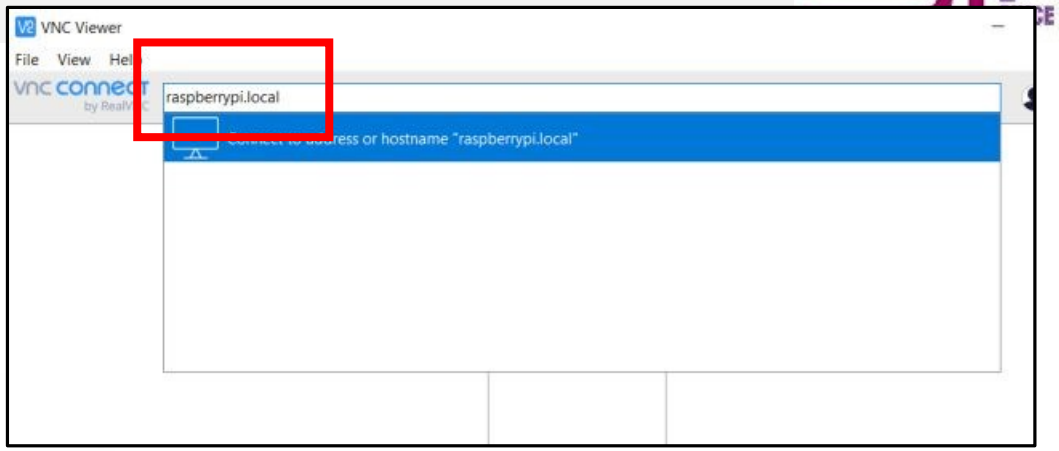
14. Move to “Finish” by pressing Tab and the Enter. It will reboot your Raspberry Pi, and the SSH session will end. Now, close Putty and other windows.

.



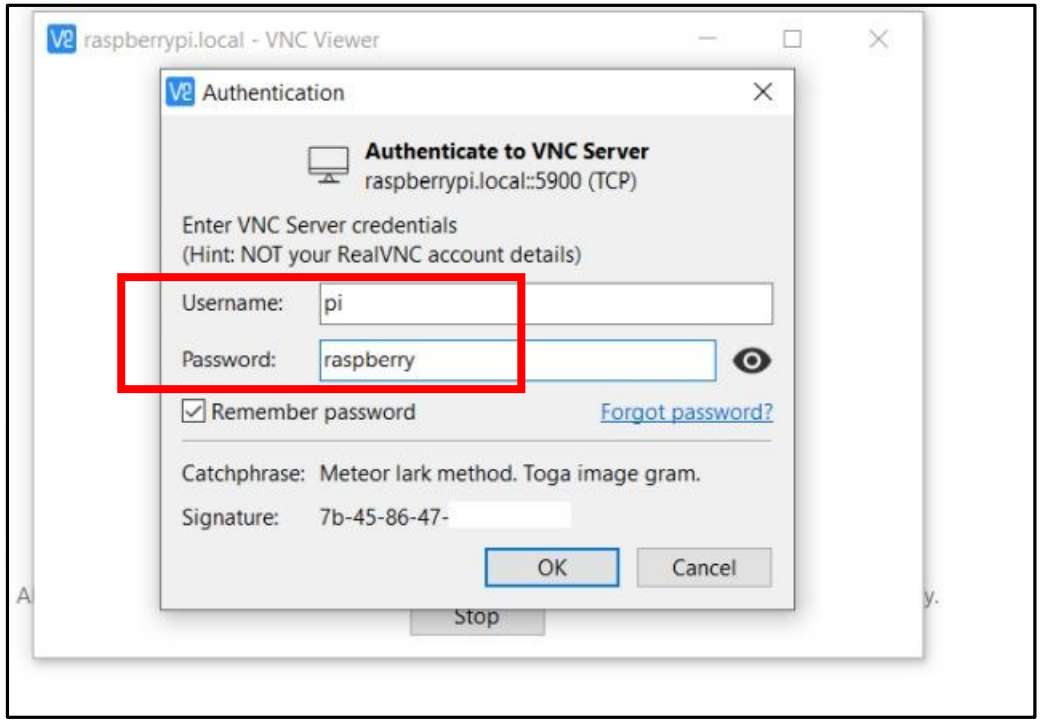
2. Access Raspberry Pi via a Computer

15. Run/open VNC Viewer. Enter raspberrypi.local and press the Enter.



16. Then, a prompt will appear, click on "Continue". On the next window, enter your RPi's credentials,
    - Username: pi
    - Password: raspberry

    Now, click on "OK".



2. Access Raspberry Pi via a Computer

17. The Raspberry Pi's screen will be live on your laptop's display. Now, go through the on-screen instructions and set up Raspberry Pi.



2. Access Raspberry Pi via a Computer

Remark:

- In addition, you can access a Raspberry Pi via a WiFi access point (WiFi communication). Note that you might need to choose the access point that you can use during the classes and exam (for example, using a mobile-phone as an access point).
  - For example, the following videos demonstrate the setups (some for MAC OS and some for Windows OS): https://www.youtube.com/results?search_query=raspberry+pi+headless+install

3. Programming GPIO Pins Using gpiozero Python Library

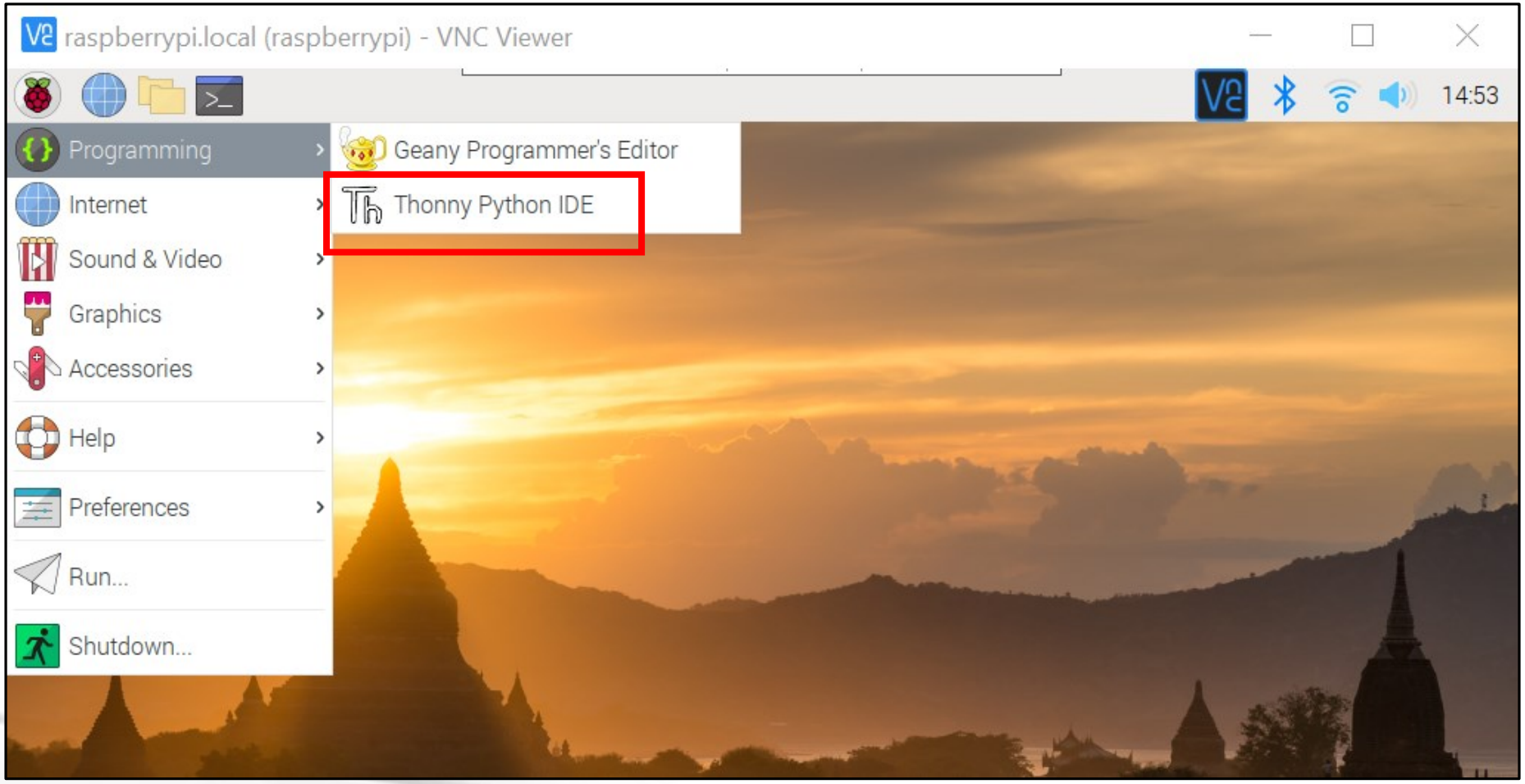
- Writing Python programs (run on a Raspbery Pi) to read inputs from switches and control LEDs, where these switches and LEDs connect to Raspberry Pi's pins.

1. Access GPIO Pins using Python
1.1 Introduction

- In this section, we will start unlocking the real power behind the Raspberry Pi - the GPIO pins, or General Purpose Input Output pins.
  - The GPIO allows you to connect your Raspberry Pi to the outside world (input/output (I/O) devices) through the use of pins that may be set to input or output, and are controlled through code.
- In this section, we will connect I/O devices (sensors/actuators) and write Python programs to interact with these I/O devices via GPIO pins.
  - Specifically, we will use LEDs and switches in our examples.
  - Since all Raspberry Pi's GPIO pins are digital pins, to read analog-input values though these pins, we will need an analog-to-digital converter chip (which we do not have). Therefore, as you will see, we will not use potentiometers here.

1. Access GPIO Pins using Python
1.2 Thonny Python IDE

We will use the Thonny Python IDE to write our Python programs. The Thonny Python IDE is included in the Raspberry Pi OS. It can be launched from the start menu under programming as shown below.



1. Access GPIO Pins using Python
1.2 Thonny Python IDE

The figure below shows the Thonny Python IDE with the classic Hello World program running with the output from the program shown at the bottom of the IDE. Note that to run a python program, you click the Run button on the menu. Note that the Python version is 3.7.3.



1. Access GPIO Pins using Python
1.3 GPIO Python Libraries

- To interact with the GPIO pins, we need to import GPIO libraries to our python programs.. There are several Python libraries which are pre-installed already in Raspberry Pi OS.
  - **RPi.GPIO**: It is a lower-level GPIO library consisting of a set of built-in functions, which are used to control, and access GPIO pins directly (similar to Arduino commands). Its functions are not aware of (not depend on) which I/O devices are being attached.
  - **PiGPIO**: It is a lower-level GPIO library.
  - **GPIOZero**: It is a high-level GPIO library. Its built-in functions are defined for most commonly used devices e.x., LED, switch, ultra-sonic sensor, etc. The functions depends on the device attached to Raspberry Pi. For a defined device, we can call its function and interact with it easily.
    - We will use this library in this course.

1. Access GPIO Pins using Python
1.4 Code Comparison

Assume that we connect an LED to the GPIO18. We want to write a Python program to turn the LED on for 5 seconds and, then, turn it off. Here is a code comparison between the Python code using the RPi.GPIO library and GPIOZero library.

RPi.GPIO Library

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setup(18, GPIO.OUT)
GPIO.output(18, GPIO.HIGH)
time.sleep(5)
GPIO.output(18, GPIO.LOW)
```

GPIOZero Library

```
from gipozero import LED
import time

led = LED(18)
led.on()
time.sleep(5)
led.off()
```

2. GPIOZero Library
2.1 Introduction

The GPIOZero python library for Raspberry Pi provides a simple interface to devices and sensors connected and controlled via GPIO pins.

- The key feature of the GPIOZero library is to provide a high-level model or interface for devices commonly used with the Raspberry Pi. Thus, unlike other libraries, it does not require the programmer to manage individual pins but provides an interface and a set of high-level methods specific to each supported component.
- For examples, the components (classes) supported by the GPIOZero library are LED, button, motor, motion sensor, etc. A list of supported components/classes is found here:

  https://gpiozero.readthedocs.io/en/stable/index.html
- In each class, a set of built-in functions (methods) is provided.
  - For example, the *led* defining as the LED class can be commanded easily by a set of built-in methods:
    - The *led.on()* and *led.off()* allow the LED to be turned on and off.
    - The *led.blink()* will let the LED turn on and off alternatively.

2. GPIOZero Library
2.1 Introduction

- In this lecture, we will experiment on 3 classes:

Class	Device	Usage
LED class	LED	To on/off an LED
PWMLED class	LED	To control the intensity of an LED
Button class	Button	To read the state (press/not press) from a button

3. LED Class
3.1 Introduction

The GPIOZero library provides a class called ***LED*** and its methods such that we can control an LED easily.

1) At the beginning of your python program, you must import this class:

```
from gpiozero import LED
```

3. LED Class
3.1 Introduction

2) Announce that the (physical) LED connecting to the GPIO *pinNum* is the class LED and call it as *led*:

led = LED(pinNum, active_high=True, initial_value=False, pin_factory=None)

- *pinNum*: It is a mandatory parameter of integer type that represents the pin number in BCM numbering to which the led is connected.
- *active_high*: It is an optional Boolean parameter (with default value *True*) that determines how the led is connected to the GPIO (i.e., *True* means that the LED is connected such that supplying 3.3V at that pin will turn the LED on).
- *initial_value*: It is an optional Boolean parameter (with default value *False*) that determines the initial state of illumination of the led (so by default the LED is off).
- *pin_factory*: It is an optional parameter for advanced use only. It allows specifying a GPIO pin factory different from the one provided by GPIOZero.
- Note that the last three can be omitted.

3. LED Class
3.1 Introduction

3) The main methods of the LED class are as follows:

Method	Meaning
on()	Turn the LED on
off()	Turn the LED off
toggle()	It reverses the lighting state of the led. If it is on it turns it off, on the contrary, if it is off it turns it on
blink(on_time=1, off_time=1, n=None, background=True)	It makes the LED blink repeatedly

We can use these methods as follows:

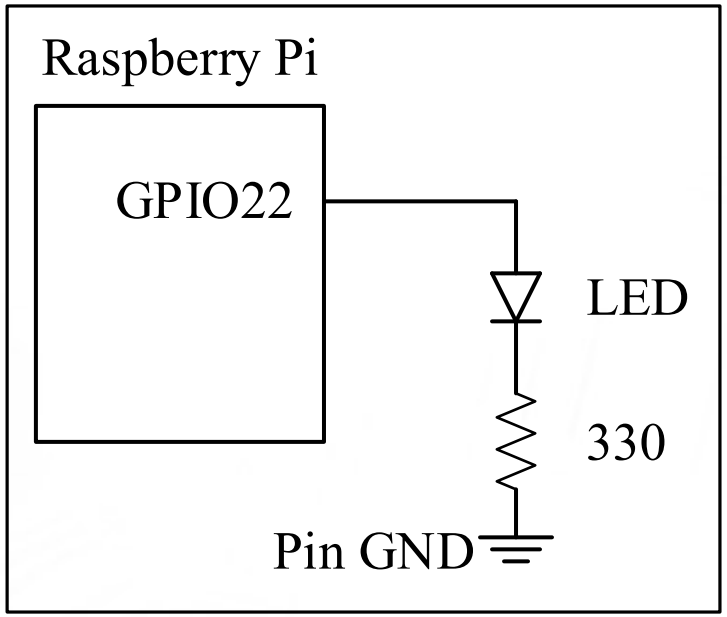
```
led.on()
led.off()
led.toggle()
led.blink(on_time=2, off_time=1)
```

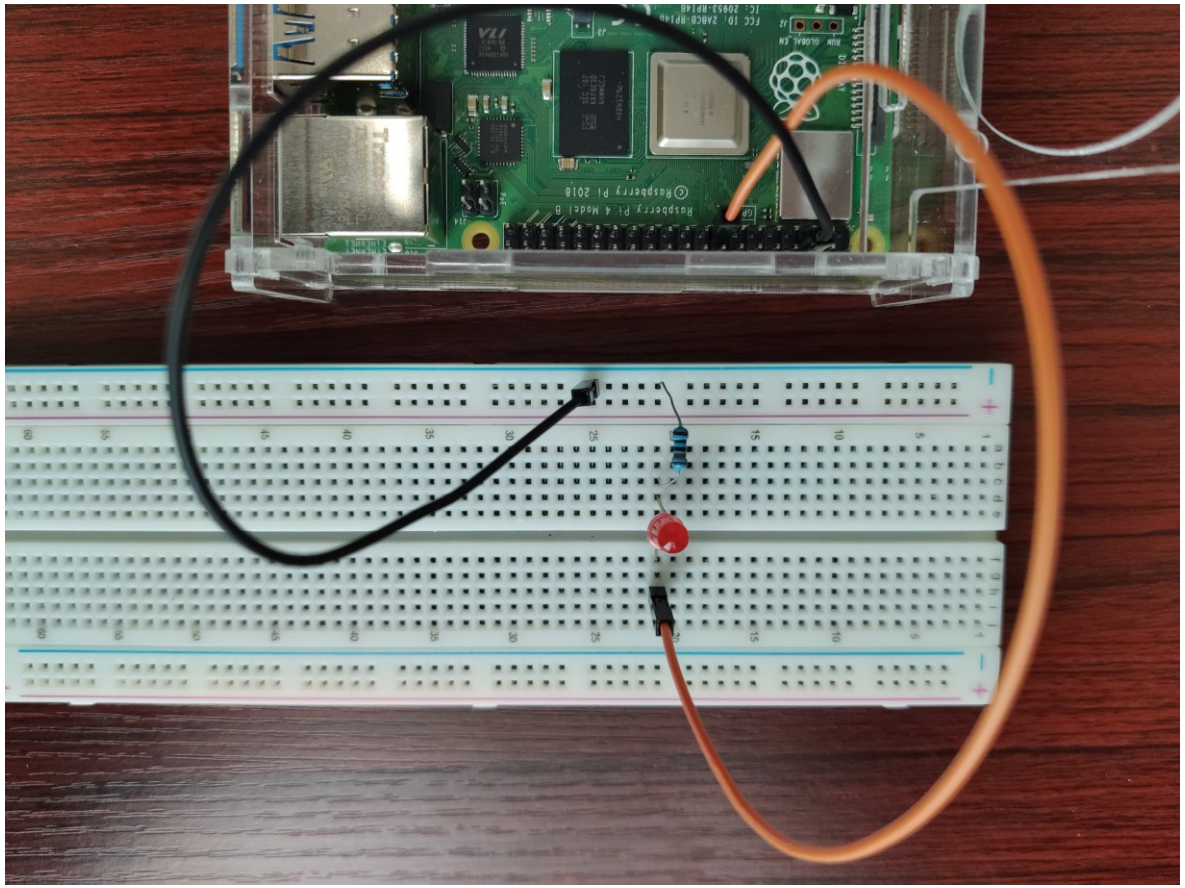
3. LED Class
3.1 Introduction

- *on_time*: it is an optional parameter, float type (with default value 1), which represents the number of seconds the led is on.
- *off_time*: it is an optional parameter, float type (with default value 1), which represents the number of seconds the led is off.
- *n*: it is an optional parameter, integer type (with default value None), which represents the number of blinks of the led. By default the led blinks infinitely.
- *background*: it is an optional parameter, of type Boolean (with default value True), that indicates if the method returns immediately to the main flow or if the method is blocking. By default, the method returns to the main stream immediately.

3. LED Class
Example 1: To Turn an LED on/off

Connect an LED to the GPIO22 and a resistor as shown below. Write a Python program by using *on()* and *off()* methods to turn the LED on and off every 2 seconds, repeatedly.





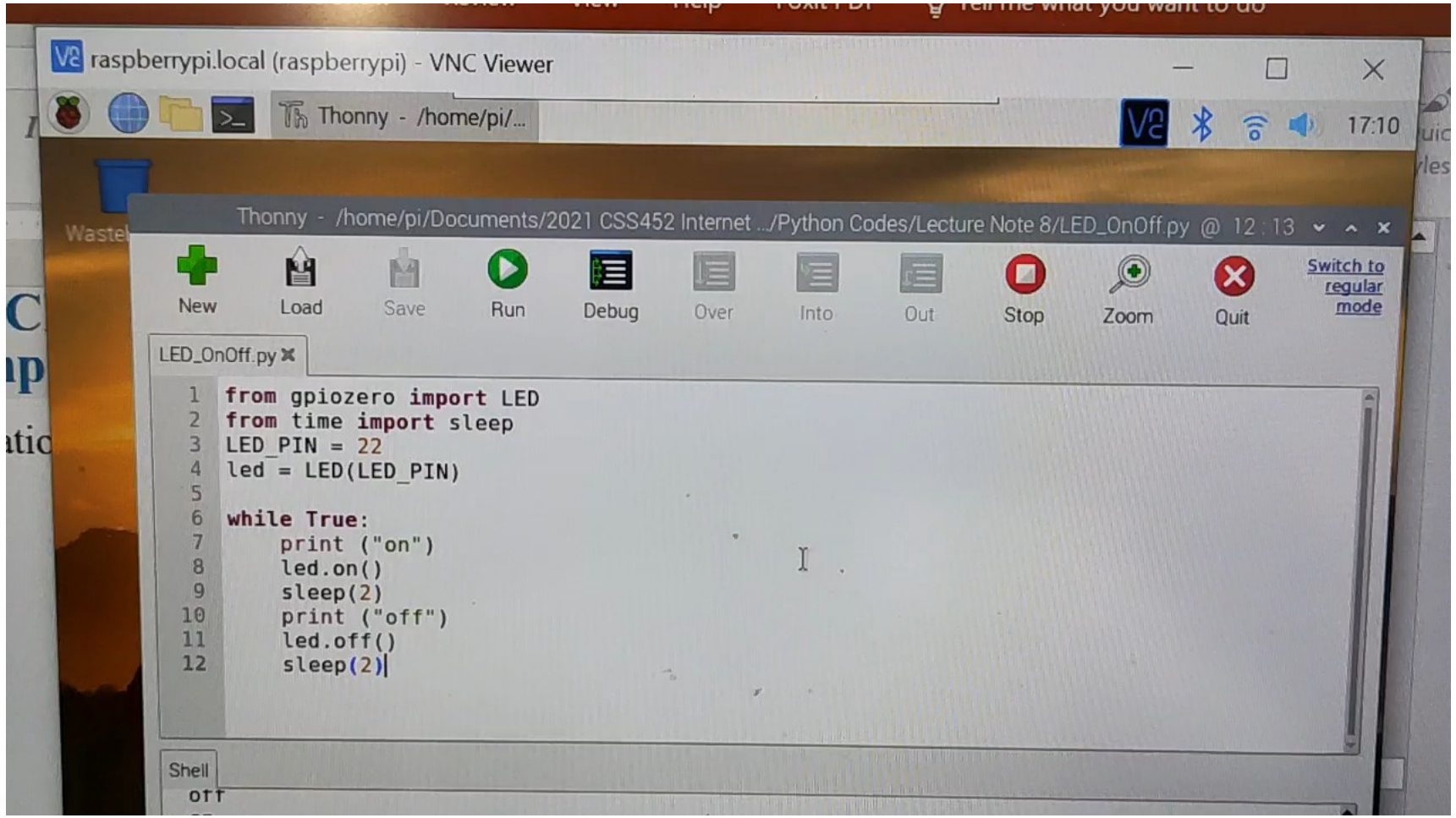
3. LED Class
Example 1: To Turn an LED on/off

Write the following Python program in the Thonny Python IDE and click the Run button to see the result.

```
1  from gpiozero import LED
2  from time import sleep
3  LED_PIN = 22
4  led = LED(LED_PIN)
5
6  while True:
7      print ("on")
8      led.on()
9      sleep(2)
10     print ("off")
11     led.off()
12     sleep(2)
```

3. LED Class
Example 1: To Turn an LED on/off

A demonstration video is shown below.



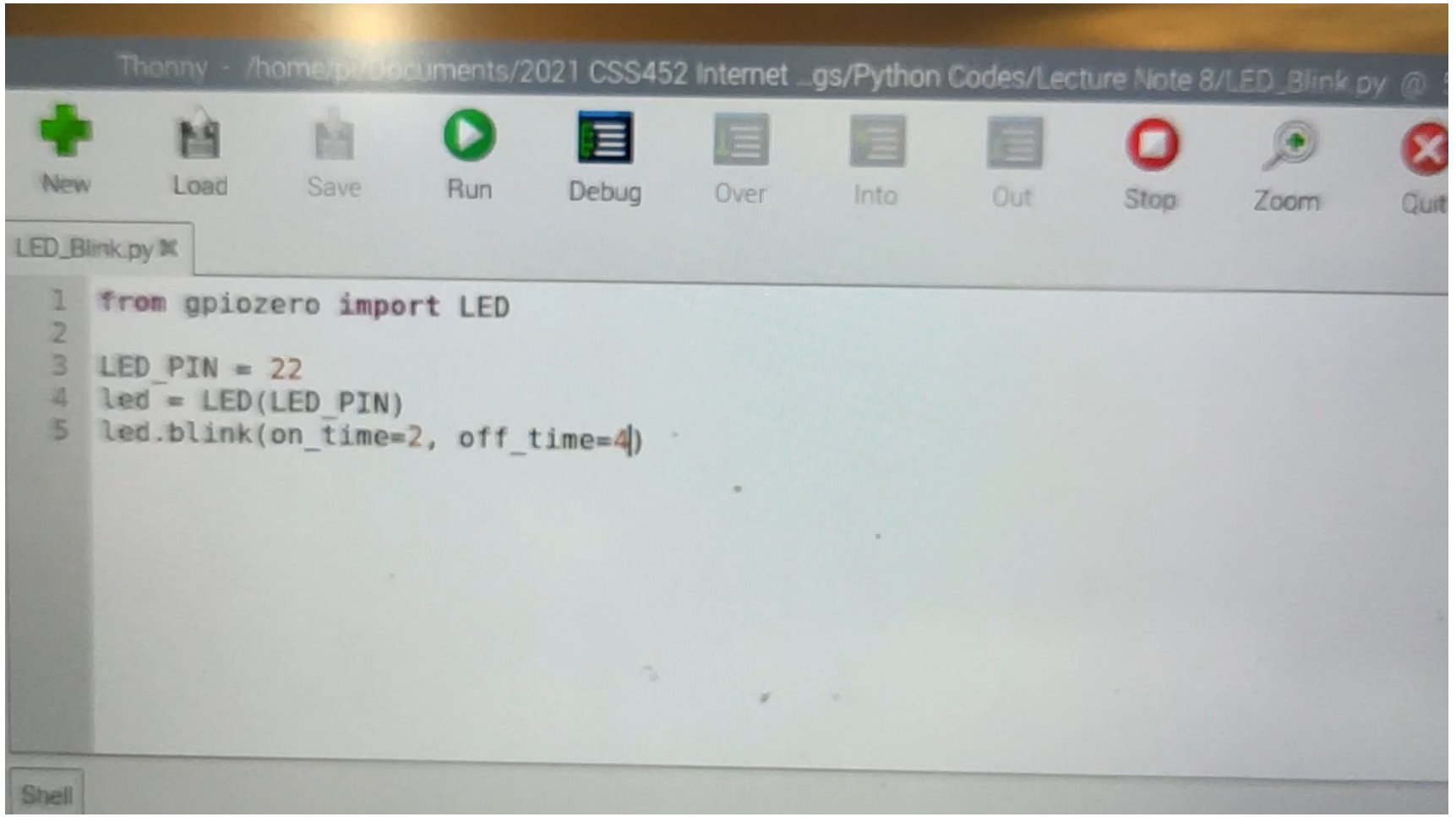
3. LED Class
Example 2: To Blink an LED

Using the same circuit connection as shown previously, write the following Python program in the Thonny Python IDE and click the Run button to see the result.

```
1  from gpiozero import LED
2
3  LED_PIN = 22
4  led = LED(LED_PIN)
5  led.blink(on_time=2, off_time=4)
```

3. LED Class
Example 2: To Blink an LED

A demonstration video is shown below.



4. PWMLED Class
4.1 Introduction

We can control the brightness/intensity of an LED by using the ***PWMLED*** class.

1) To import this class into your Python program, write this statement:

```
from gpiozero import PWMLED
```

2) Define the (physical) LED connecting to the *pin* of the Raspberry Pi as the *PWMLED* class:

```
led = PWMLED(pin)
```

where *pin* is a mandatory parameter of integer type that represents the pin number in BCM numbering to which the led is connected.

3) Use the following command to specify the voltage supply to the *pin*:

```
led.value = Vnum
```

where *Vnum* is a float number between 0 (off) and 1 (highest voltage).

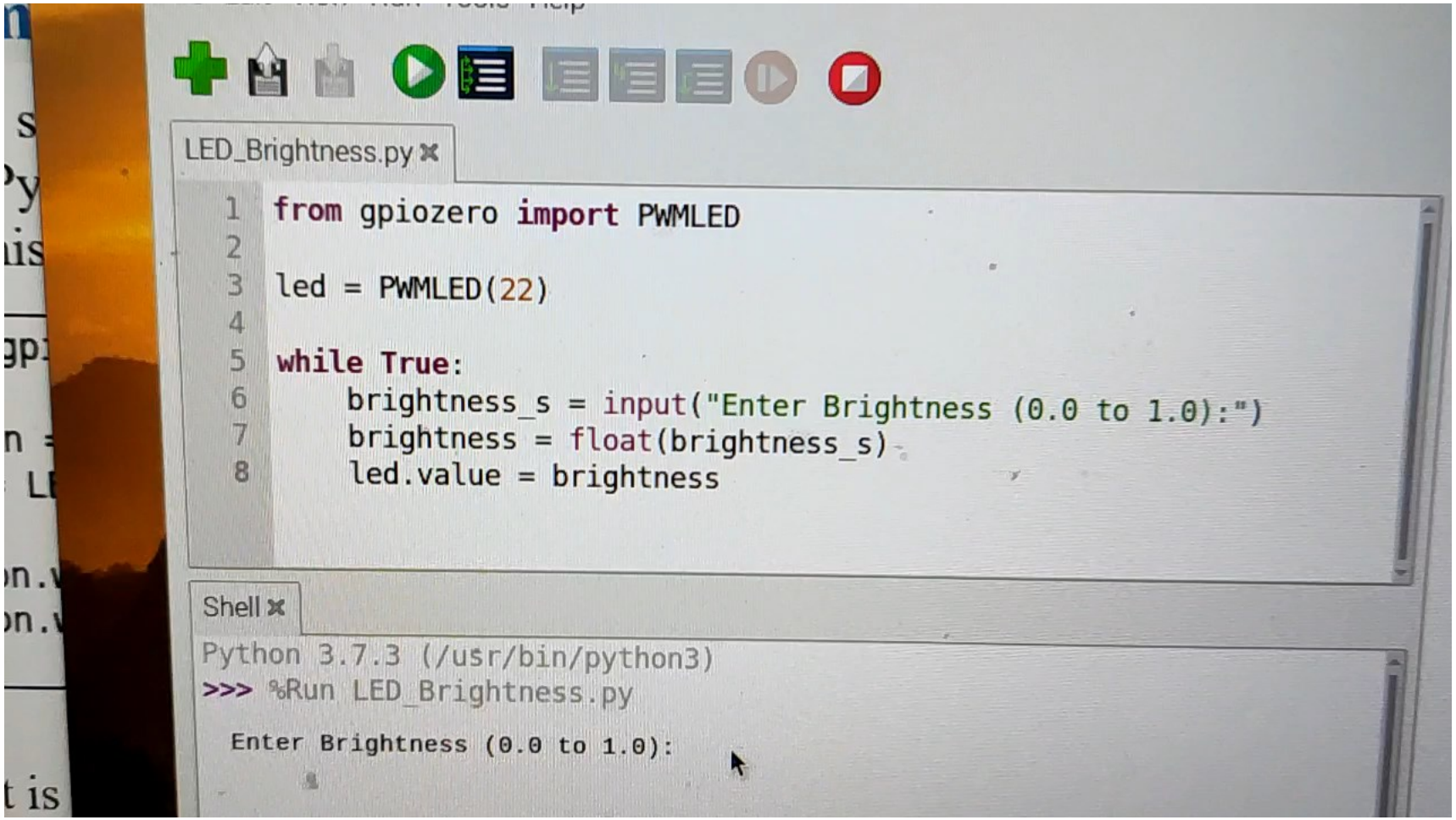
4. PWMLED Class
Example 3: Adjust LED Brightness

Connect an LED to the GPIO22 and a resistor as shown in the previous example. Write a Python program to receive an input value and adjust the LED intensity according to the input value.

```
1  from gpiozero import PWMLED
2
3  led = PWMLED(22)
4
5  while True:
6      brightness_s = input("Enter Brightness (0.0 to 1.0):")
7      brightness = float(brightness_s)
8      led.value = brightness
```

4. PWMLED Class
Example 3: Adjust LED Brightness

A demonstration video is shown below.



5. Button Class
5.1 Introduction

There are two possible button configurations according to the position of the resistor:



a) **Pull-Up**: when a resistor (10-100K Ohm) is inserted between the GPIO and the power supply (VCC).
   - When the button is not pressed (OFF) the pin assumes stable logic state 1 (HIGH). Contrary to the case in which the resistor is not present the logic state 1 remains at the GPIO input of the Raspberry Pi in a stable and permanent way. Once the button is pressed the current flows to the ground bringing the pin to logic state 0 (LOW).

5. Button Class
5.1 Introduction

b) **Pull-Down**: when a resistor (10-100K Ohm) is inserted between the GPIO and ground (GND).
   - With the OFF button (not pressed) the GPIO assumes stable logic state 0 (LOW). Since the resistor is present the logic state 0 remains at the input of the Raspberry Pi pin without oscillations. In this case when the button is pressed the current flows to the ground bringing the pin to logic state 1 (HIGH).

5. Button Class
5.1 Introduction

The ***Button*** within the GPIOZero library belongs to the Input device class and is modeled by a simple intuitive interface.

1) At the beginning of your python program, you must import this class:

```
from gpiozero import Button
```

5. Button Class
5.1 Introduction

2) The constructor of the class has the following syntax:

button = Button(pin, pull_up=True, bounce_time=None, hold_time=1, hold_repeat=False, pin_factory=None)

- *pin:* Mandatory parameter of integer type, it represents the pin number in BCM numbering to which the button is connected.
- *pull_up*: Optional Boolean parameter, it enables the internal *Pull-Up* (True - Default or Pull-Down (False) resistor.
- *bounce_time*: Optional parameter of float type, it avoids software bounces by specifying the number of seconds in which changes in the state of the button are ignored, by default it is disabled.
- *hold_time*: Optional parameter of float type, it specifies the duration in seconds of the button pressure after which the *when_held* handler is executed.
- *hold_repeat*: Optional parameter of Boolean type, if *True* (Default) every *hold_time* seconds it calls the *when_held* handler, if *False* it calls it only once.
- *pin_factory*: Optional parameter for advanced use only, it allows specifying a GPIO pin factory different from the one provided in GPIOZero.

5. Button Class
5.1 Introduction

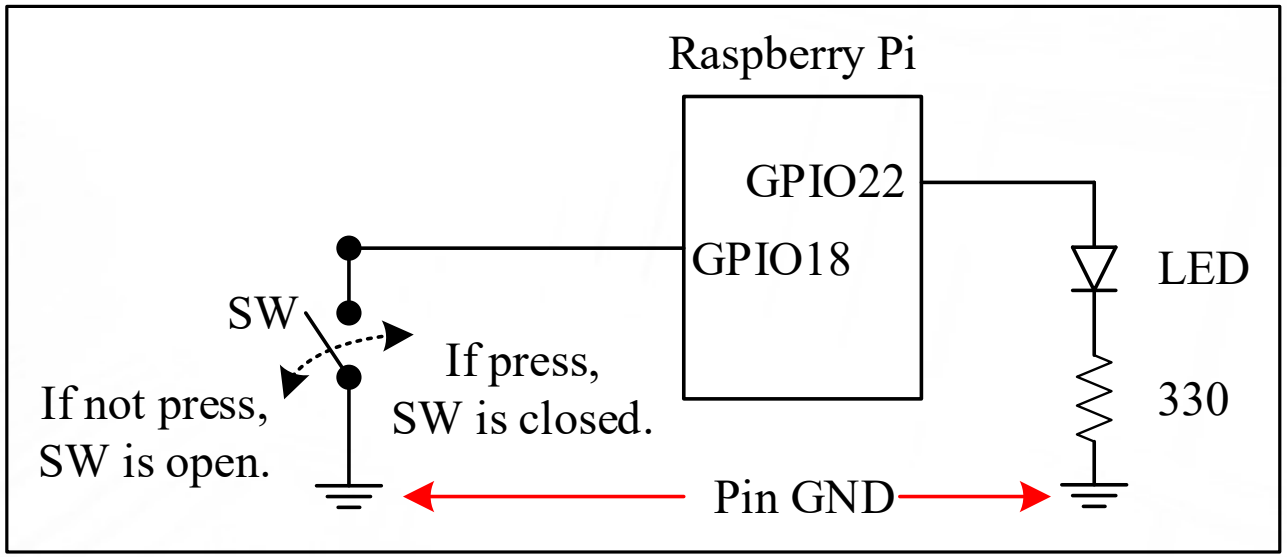
3) The main methods of the Button class are as follows:

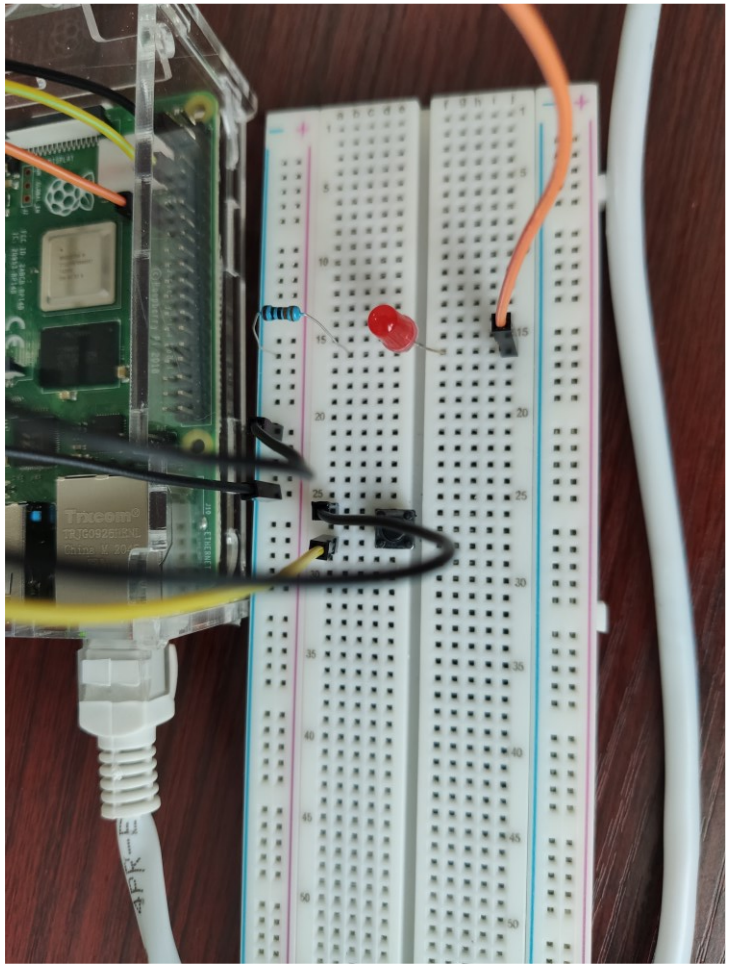
Method	Meaning
is_pressed	Return *TRUE* if the button is pressed.
wait_for_press(Timeout=None)	Blocking call that waits for the button to be pressed or for the float type Timeout to expire.
wait_for_release(Timeout=None)	Blocking call that waits for the release of the button or the expiration of the float type Timeout.
when_held	It performs the associated function whenever the button is held down for *hold_time* seconds (*hold_time* is defined in the previous step).
when_pressed	It performs the associated function whenever the button is pressed.
when_released	It performs the associated function whenever the button is released.

We can use these methods as follows: *button.is_pressed*

5. Button Class
Example 4: To Turn an LED on/off by a Switch (1)

Connect an LED to the GPIO22 and a resistor as shown below (the pull-up configuration). Connect a switch to the GPIO18 as shown below (no resistor is needed). Write a Python program such that pressing the switch turn the LED on; otherwise, the LED is off.





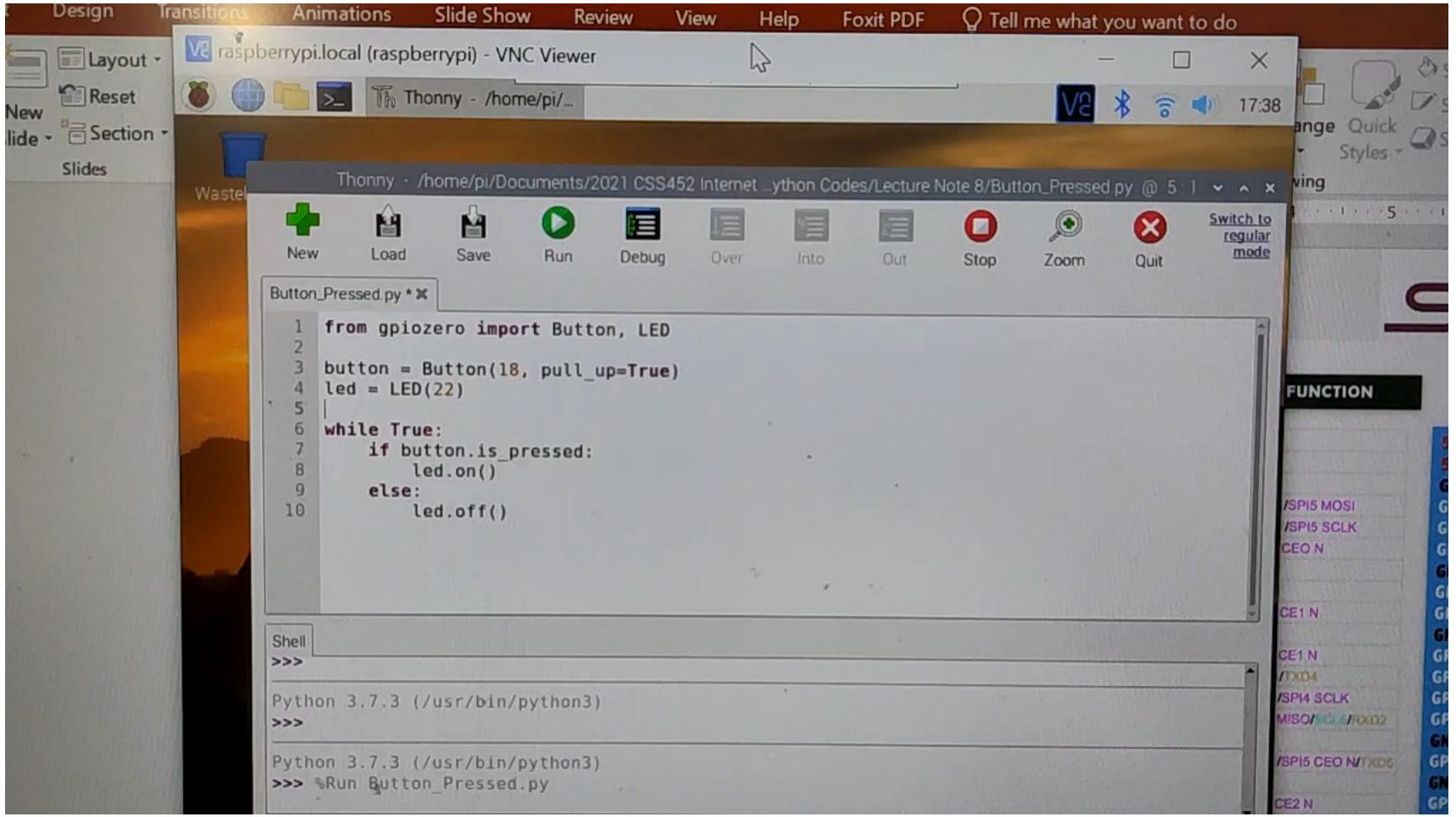
5. Button Class
Example 4: To Turn an LED on/off by a Switch (1)

Write the following Python program in the Thonny Python IDE and click the Run button to see the result. In this example, we will use *is_pressed* method (which will return *True* when we press the switch).

```
1  from gpiozero import Button, LED
2
3  button = Button(18, pull_up=True)
4  led = LED(22)
5
6  while True:
7      if button.is_pressed:
8          led.on()
9      else:
10         led.off()
```

5. Button Class
Example 4: To Turn an LED on/off by a Switch (1)

A demonstration video is shown below.



5. Button Class
Example 5: To Turn an LED on/off by a Switch (2)

Connect a switch and an LED to the Raspberry Pi as shown previously. Write the following Python program in the Thonny Python IDE and click the Run button to see the result. In this example, we will use:

- *when_pressed* method (which will run the function on the right-hand side of the equal sign when we press the switch).
- *when_released* method (which will run the function on the right-hand side of the equal sign when we press the switch).

```
1  from gpiozero import Button, LED
2
3  button = Button(18, pull_up=True)
4  led = LED(22)
5
6  button.when_pressed = led.on
7  button.when_released = led.off
```

The result is similar to the previous example.

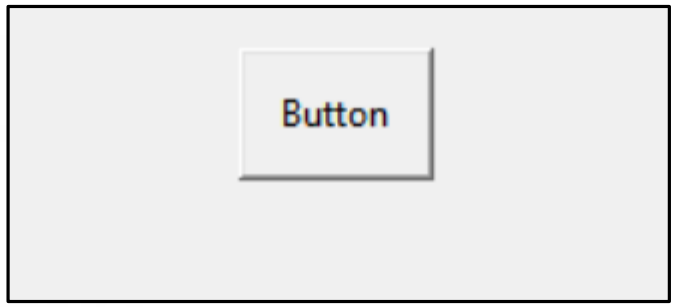
4. Creating a Simple GUI using guizero Python Library

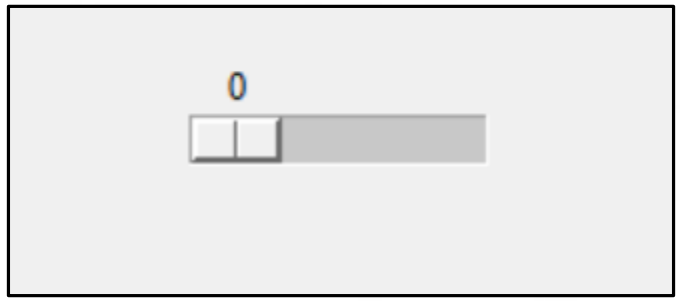
- Writing Python programs (run on a Raspbery Pi) to create simple GUIs and control LEDs, where these LEDs connect to Raspberry Pi's pins.

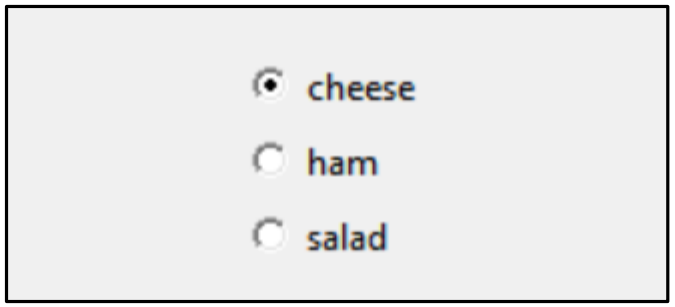
1. Introduction

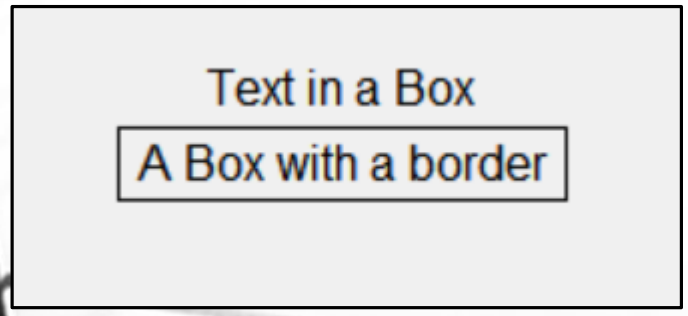
guizero is a Python 3 library for creating simple GUIs. It is designed to allow new learners to quickly and easily create GUIs for their programs. It provides many built-in functions to create different kinds of widgets (such as button, slider, etc.). A list of available widgets is shown here:

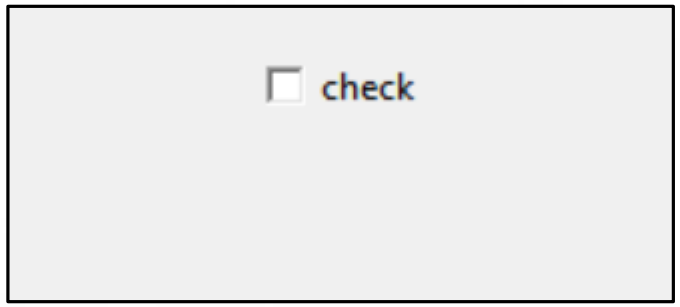
https://lawsie.github.io/guizero/widgetoverview/









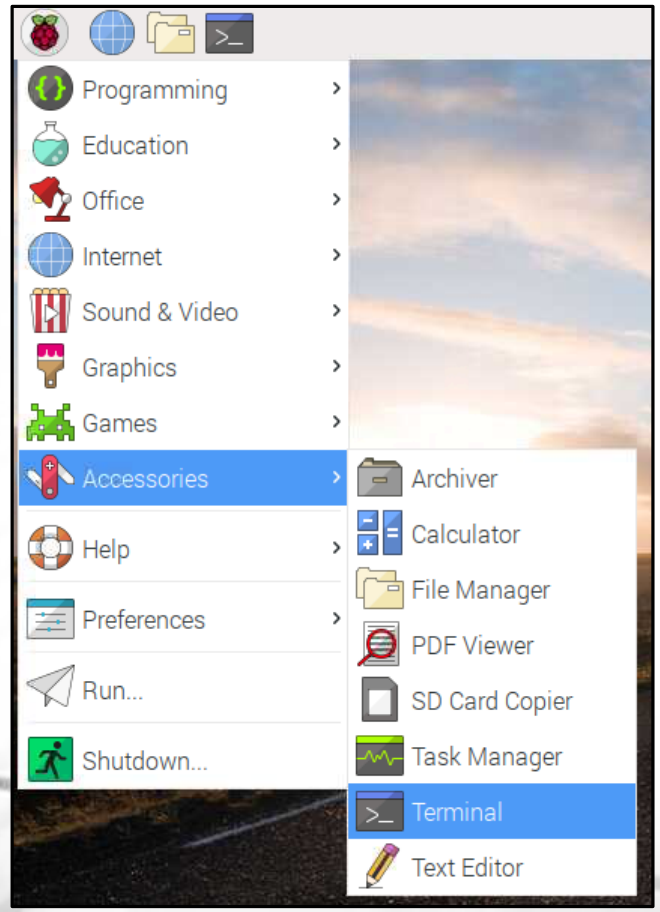




2. Install the guizero Library

Before we can use it in our Python programs, we need to install the guizero library as follows.

1) Open a terminal window by clicking Menu > Accessories > Terminal.
2) Type this command on the Terminal and press enter:



sudo pip3 install guizero

or, if error, we might use this one instead:

sudo apt install python3-guizero

or follow the suggestion provided in the Terminal.

3. Code Examples of using the guizero Library

1) **App object**: The App object is the basis of all GUIs created using guizero. It is the main window which contains all of the other widgets. To create an App object, we use the command:

app = App()

where *app* is the variable name of the App object. To display the window of this App object, we use the command:

app.display()

For example, we can use this code to create a window of an App object:

```
from guizero import App
app = App()
app.display()
```

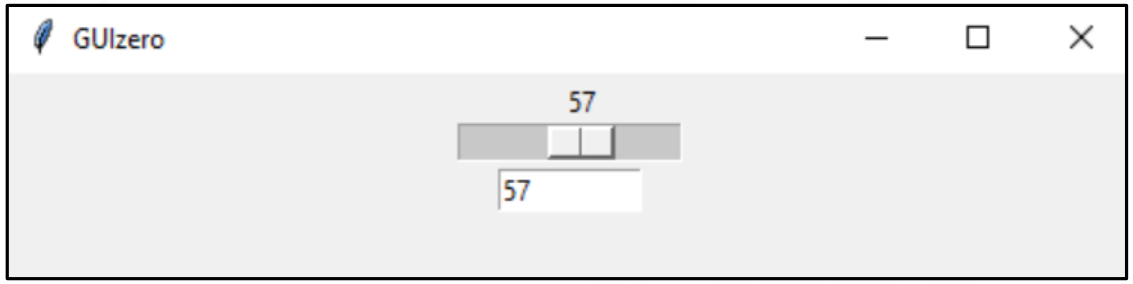


3. Code Examples of using the guizero Library

2) **Slider object**: The Slider object displays a bar and selector which can be used to specify a value in a range 0-100. The command is:

slider = Slider(app, command=NameOfFunction)

- *slider* is the variable name of the Slider object, *app* is the variable name of the App object.
- *NameOfFunction* is a custom function to be run when we slide the slider bar.



3. Code Examples of using the guizero Library

3) **Text Box object**: The Text Box object create a text box and will display a value inside. The command is:

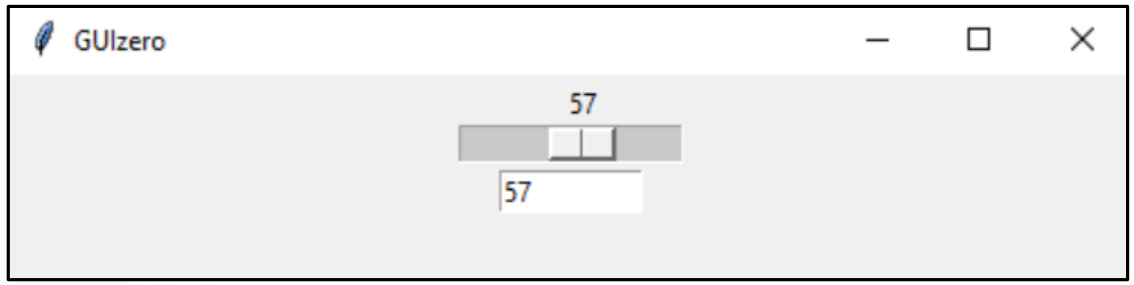
textbox = TextBox(app)

- *textbox* is the variable name of the TextBox object.
- *app* is the variable name of the App object.

To specify the value shown in the text box, we use the command

textbox.value = ValueToBeShown

- The parameter *ValueToBeShown* is the value that we want to display in the text box.



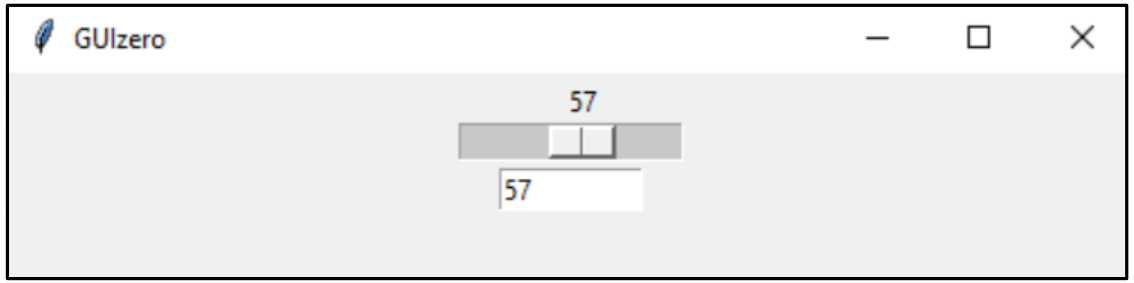
Example: Slider and Text Box

For example, the Python code below will create a Slider object and a Text Box object.

- Obj: The current value of the Slider object (between 0 and 100) will be displayed in the Text Box object. We change the Slider object's value by moving the Slider button.

```
from guizero import App, Slider, TextBox
def slider_changed(slider_value):
    textbox.value = slider_value

app = App()
slider = Slider(app, command=slider_changed)
textbox = TextBox(app)
app.display()
```



- We write a custom function "*slider_changed(slider_value)*" to be called every time when we change the Slider object's value.
- The Slider object's value will be automatically passed to the input argument "*slider_value*" of the custom function "*slider_changed*".

3. Code Examples of using the guizero Library

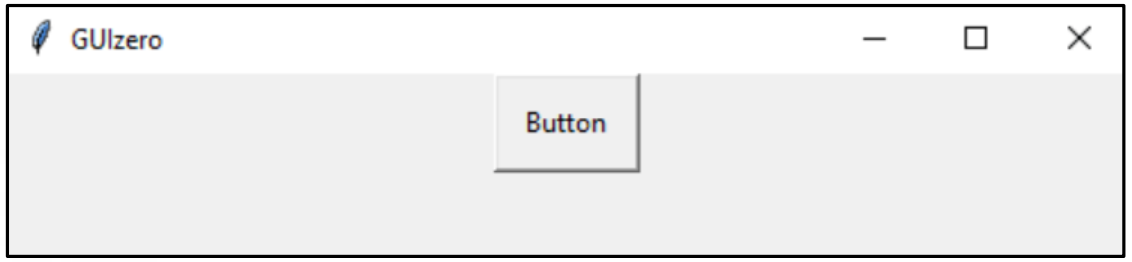
4) **Push Button object**: The Push Button object displays a button with text or an image, which calls a function when pressed. We can create it by the following Python command:

button = PushButton(app, command= NameOfFunction)

- *button* is the variable name of the Push Button object, *app* is the variable name of the App object.
- *NameOfFunction* is a custom function to be run when we push the button. For example, we can use this code to create a Push Button object:

```
from guizero import App, PushButton
def do_nothing():
    print("Button was pressed")

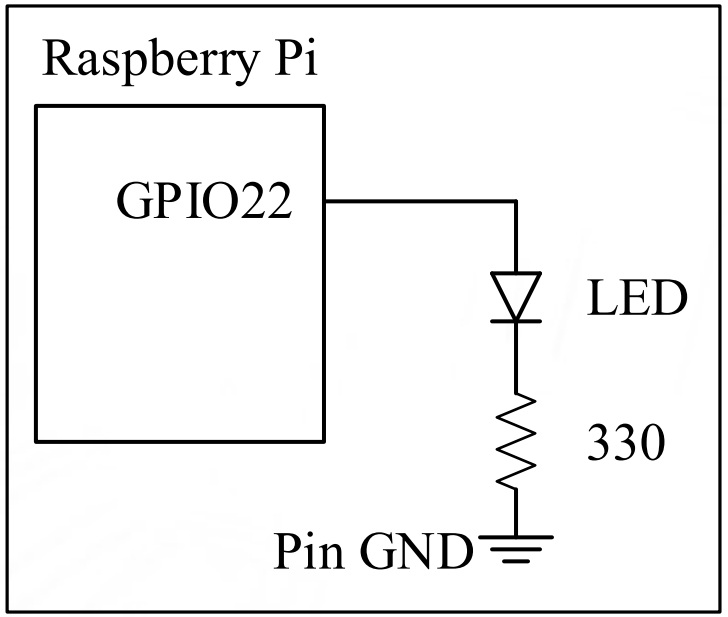
app = App()
button = PushButton(app, command=do_nothing)
app.display()
```

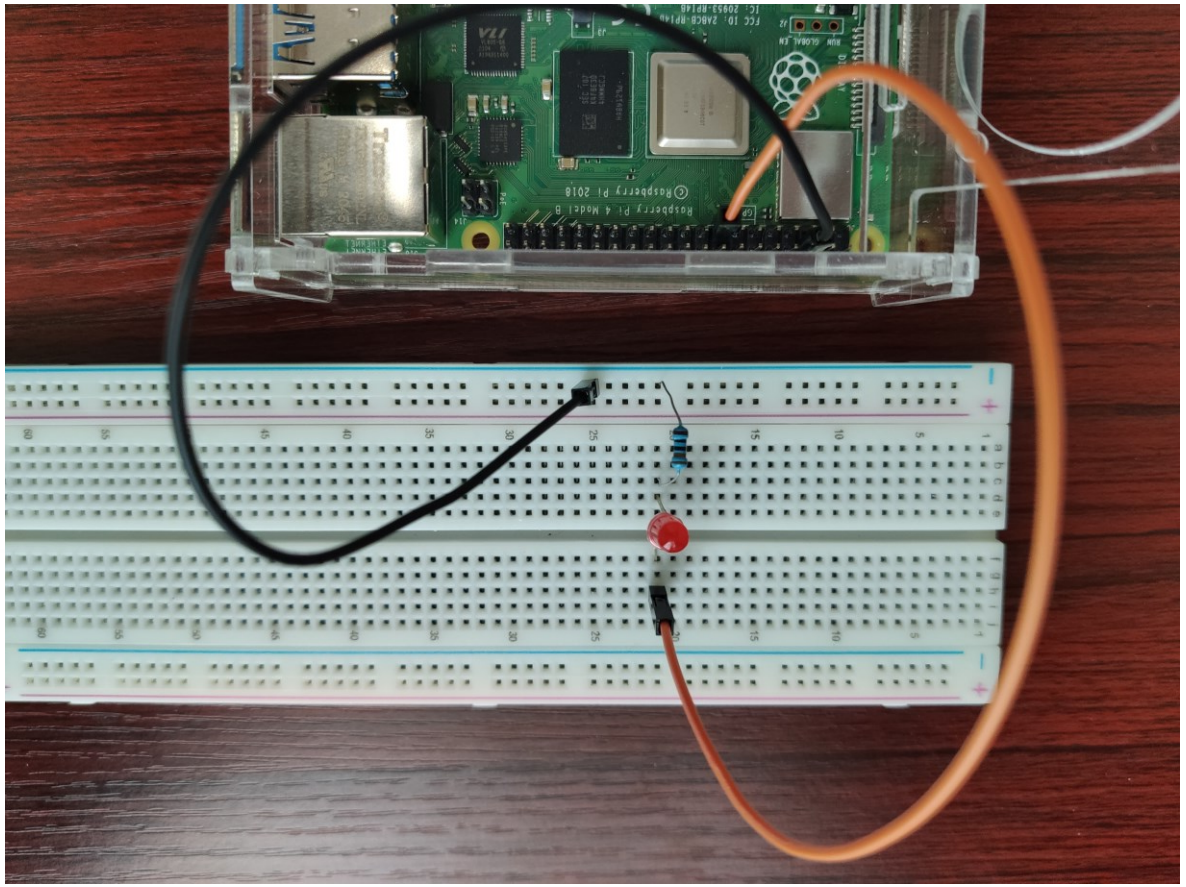


Example 6

Connect an LED to the GPIO22 an a resistors as shown below.

- Write a Python program to create two push buttons (GUI by using the guizero library).
- Pressing these buttons will turn the LED on/off.





Example 6

The Python code is shown below. More explanations of the methods used in this code can be found here:

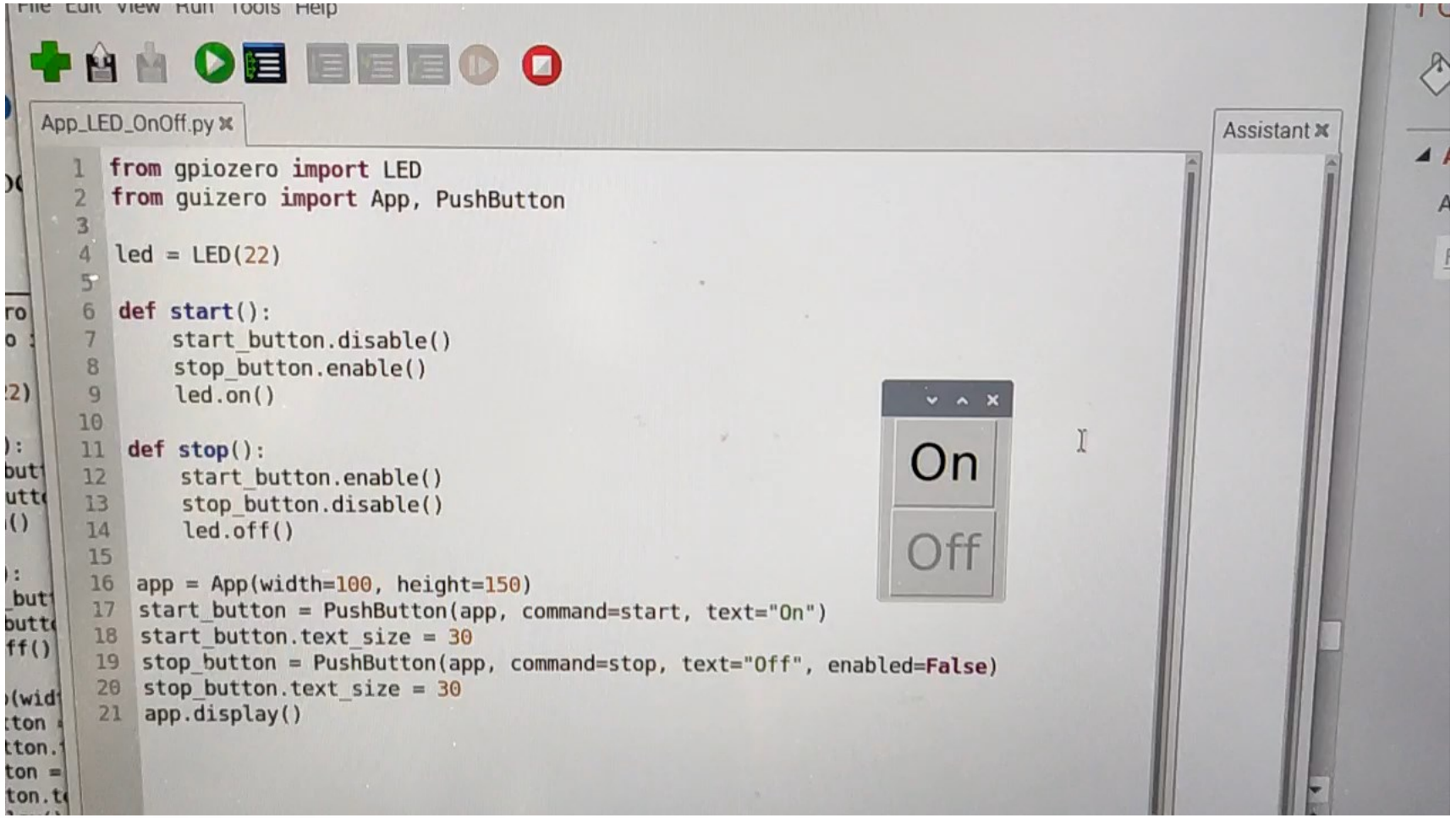
https://lawsie.github.io/guizero/pushbutton/

```
1  from gpiozero import LED
2  from guizero import App, PushButton
3
4  led = LED(22)
5
6  def start():
7      start_button.disable()
8      stop_button.enable()
9      led.on()
10
11 def stop():
12     start_button.enable()
13     stop_button.disable()
14     led.off()
15
16 app = App(width=100, height=150)
17 start_button = PushButton(app, command=start, text="On")
18 start_button.text_size = 30
19 stop_button = PushButton(app, command=stop, text="Off", enabled=False)
20 stop_button.text_size = 30
21 app.display()
```



Example 6

A demonstration video is shown below.



Example 7

Connect an LED to the GPIO22 and a resistor as shown in the previous example (Example 6). Write a Python program to create a slider (GUI by using the guizero library). Sliding it will adjust the brightness/intensity of the LED. The Python code is shown below.

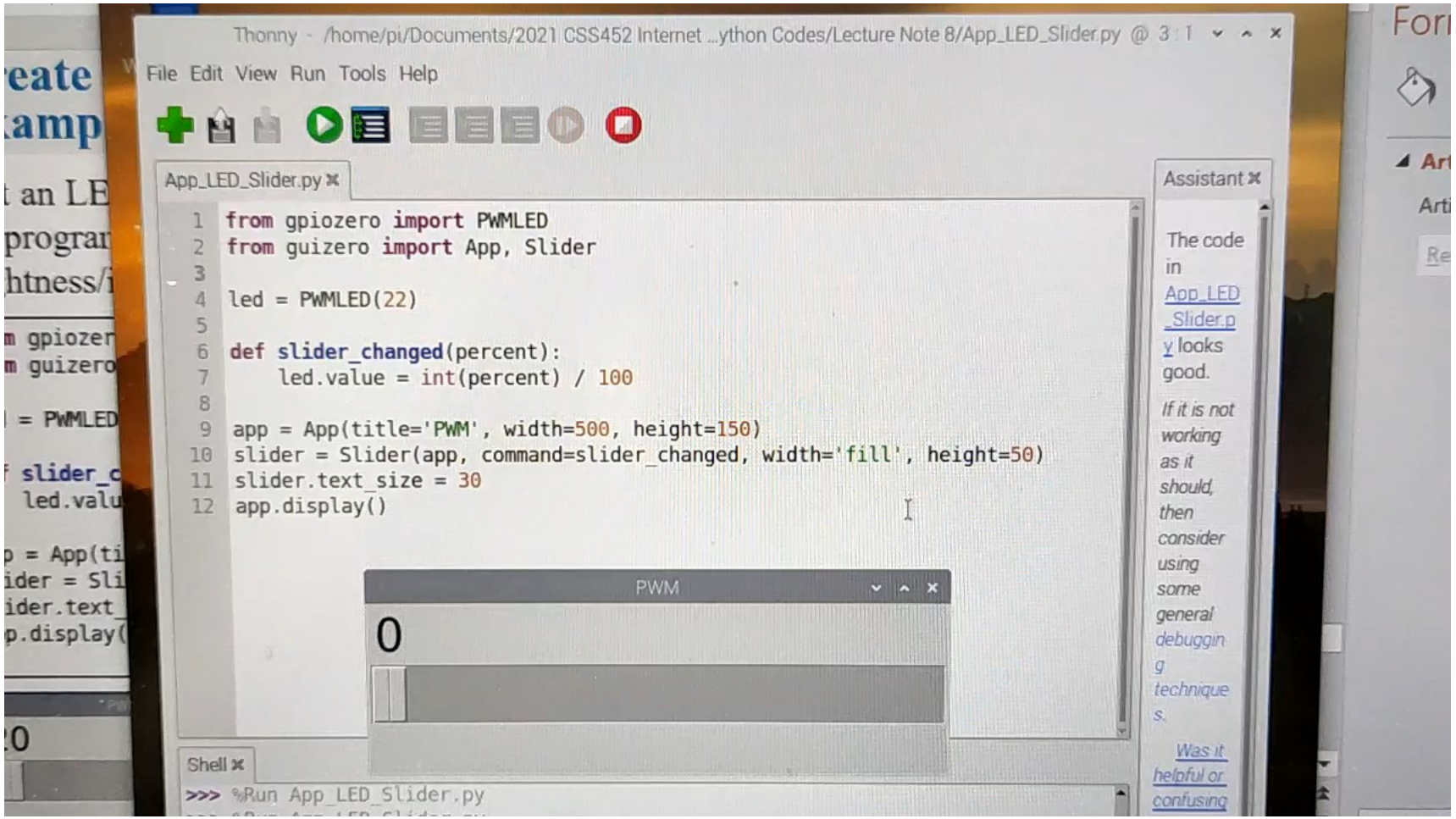
```
1  from gpiozero import PWMLED
2  from guizero import App, Slider
3
4  led = PWMLED(22)
5
6  def slider_changed(percent):
7      led.value = int(percent) / 100
8
9  app = App(title='PWM', width=500, height=150)
10 slider = Slider(app, command=slider_changed, width='fill', height=50)
11 slider.text_size = 30
12 app.display()
```



More explanations can be found here:
https://lawsie.github.io/guizero/slider/

Example 7

A demonstration video is shown below.



5. Programming Raspberry Pi Towards IoT Applications Using Python

Introduction

The Internet of Things (IoT) is the rapidly growing network of devices (things) connected to the internet. That does not just mean more and more computers using browsers, but actual appliances and wearable and portable technology. This includes all sorts of home automation, from smart appliances and lighting to security systems and even internet-operated pet feeders, as well as lots of less practical but fun projects.

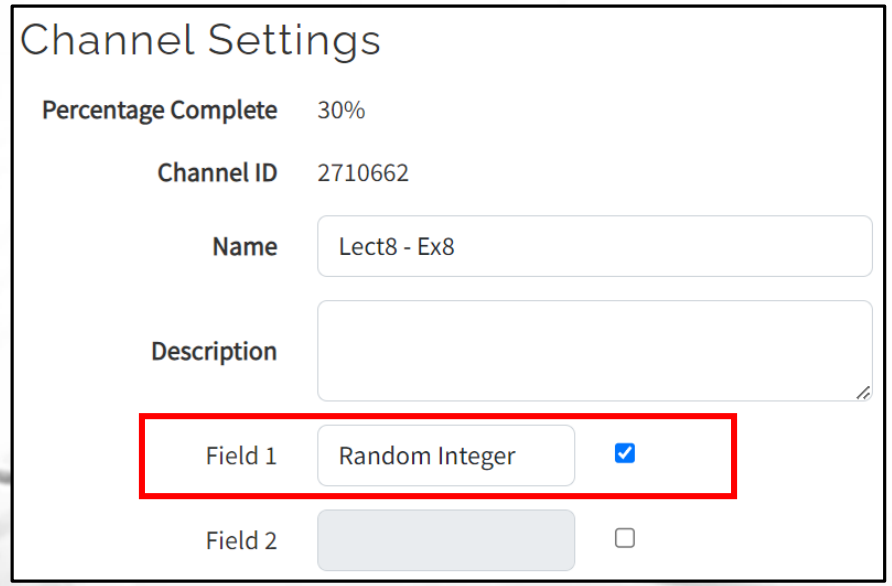
In this section, we will study an examples on how our Raspberry Pi can participate in the IoT using Python.

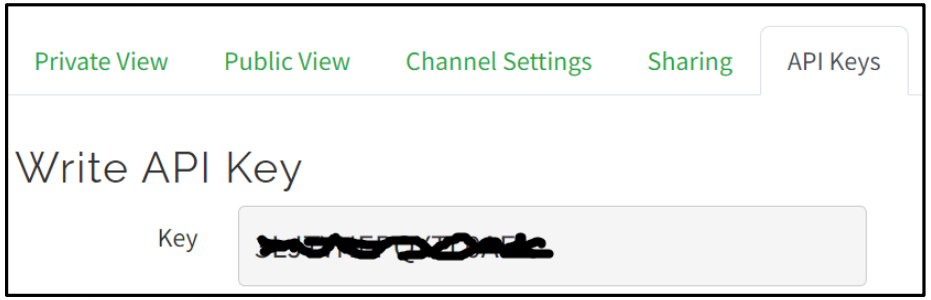
Example 8

In this example, we will write a Python code to send a random integer to the ThingSpeak dashboard every one minute. No circuit connection is required. However, we have to set up a channel in the ThingSpeak as we have done in Lecture Note 5 and get its API Key.

- In your ThingSpeak, create a new channel as follows:
  - Name: Lect8 – Ex8
  - Field 1: Random Integer

  Other parameters are left blank. Press the Save Channel button to finish it.
- Click in the API Keys tab and record your Write API Key for this channel.



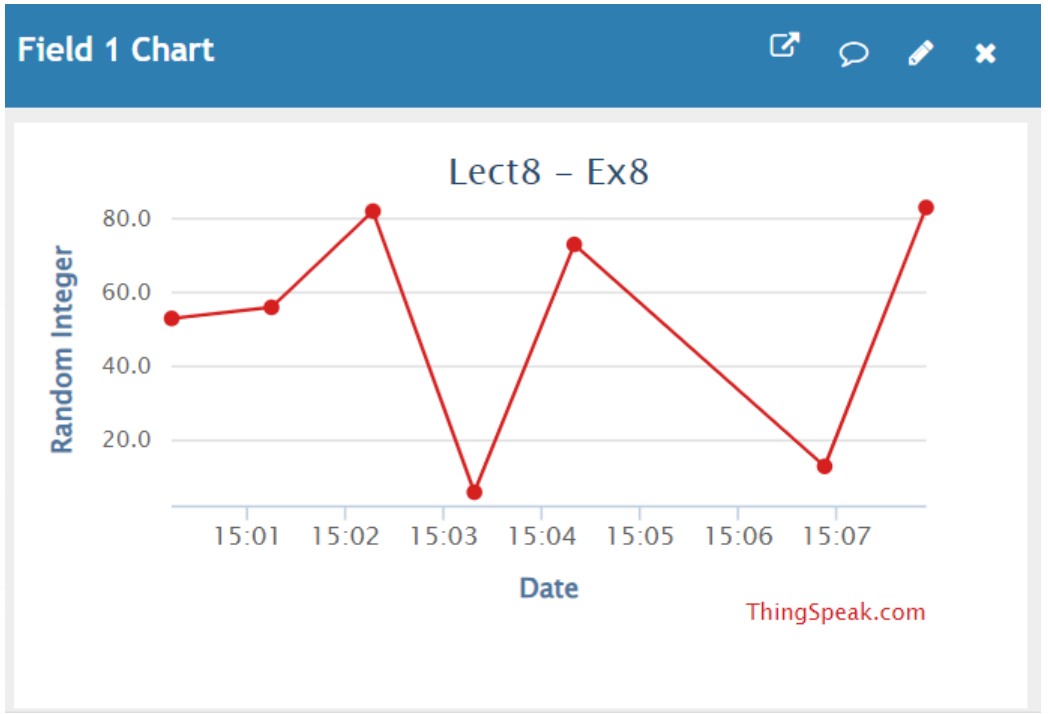


Example 8

The Raspberry Pi must connect to the Internet.

Running the Python code below will send a random integer (0-100) to the ThingSpeak. We can see the values shown on the graph in the ThingSpeak. Note that put your own Write API Key.

```
1   import time, requests
2   from random import randint
3
4   PERIOD = 60 #seconds
5   BASE_URL = 'https://api.thingspeak.com/update.json'
6   KEY = 'Your Write API Key'
7
8   def send_data(Value):
9       data = {'api_key': KEY, 'field1': Value}
10      response = requests.post(BASE_URL, json=data)
11
12  while True:
13      Number = randint(0, 100)
14      print('Number: ' + str(Number))
15      send_data(Number)
16      time.sleep(PERIOD)
17
```



Questions?